

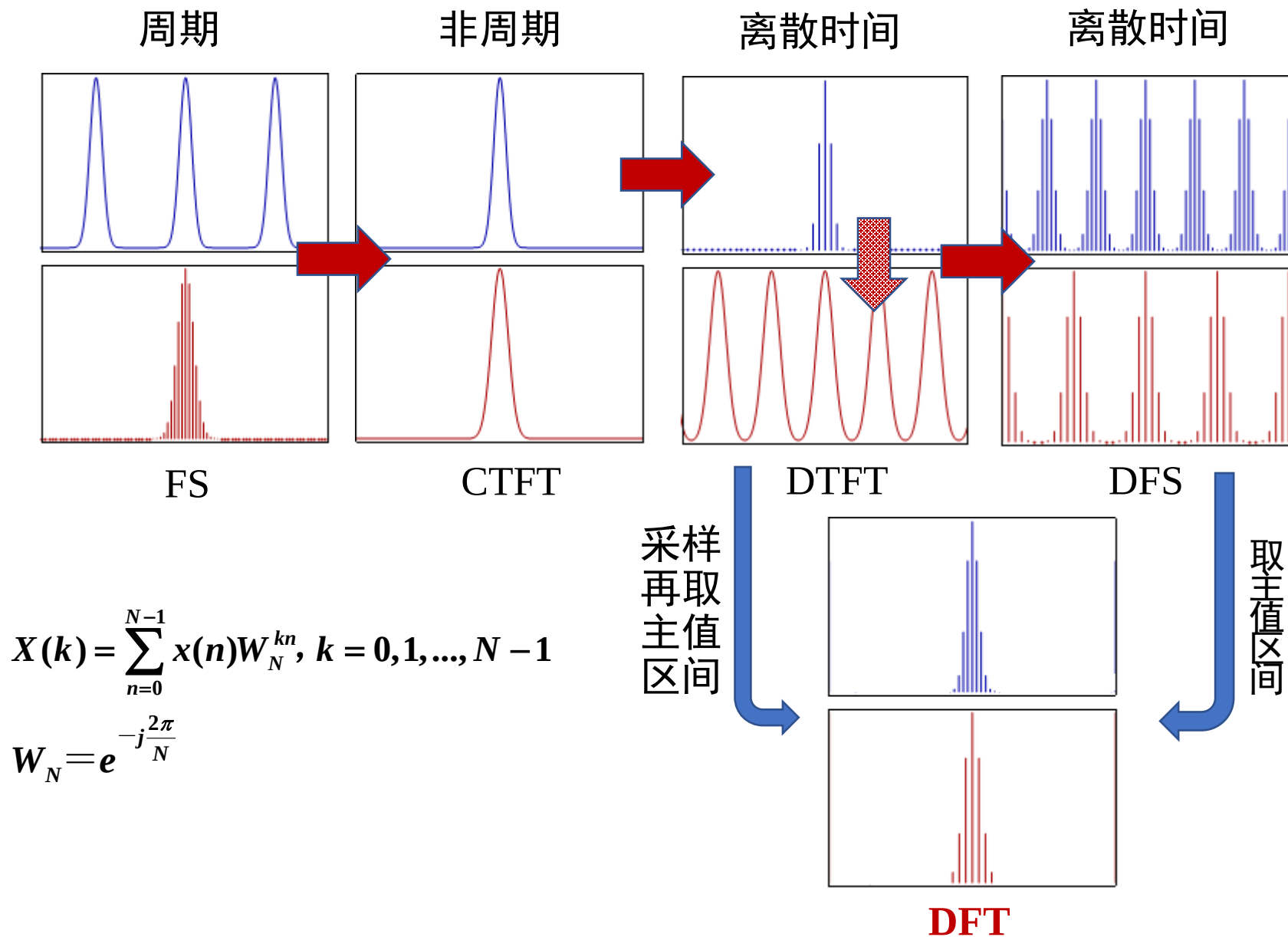
信号处理原理-10

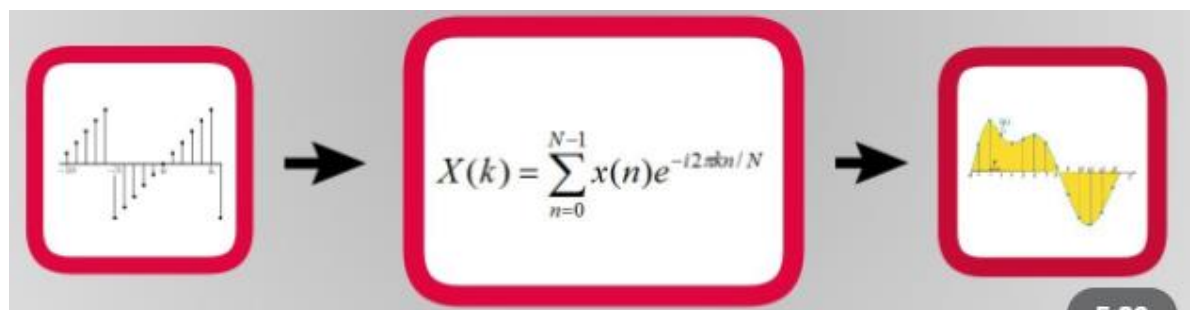
刘华平

清华大学

之前的一切都是在为DFT做铺垫

1





$$\begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} = \begin{bmatrix} W^0 & W^0 & W^0 & \dots & W^0 \\ W^0 & W^{1 \times 1} & W^{2 \times 1} & \dots & W^{(N-1) \times 1} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ W^0 & W^{1 \times (N-1)} & W^{2 \times (N-1)} & \dots & W^{(N-1) \times (N-1)} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}$$

本节课，可以忘记频率，只需要操作序列、数列

离散傅里叶变换 问题与特点

➤DFT的计算量分析

N 点长序列 $x(n)$

正变换：

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{nk} \quad k = 0, 1, \dots, N-1$$

反变换：

$$x(n) = \text{IDFT}[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k)W_N^{-nk} \quad n = 0, 1, \dots, N-1$$

DFT正反变换的运算方式和运算量是相同的，后面的分析以DFT正变换为例。

DFT问题与特点

5

➤ DFT的计算量分析

N 点长序列 $x(n)$

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad k = 0, 1, \dots, N-1$$

$$X(k) = \sum_{n=0}^{N-1} \cdot$$

$N-1$ 次加法

$$\underset{1}{x(0)} \cdot e^{-j\frac{2\pi}{N}0k} + \underset{2}{x(1)} \cdot e^{-j\frac{2\pi}{N}1k} + \dots + \underset{N}{x(N-1)} \cdot e^{-j\frac{2\pi}{N}(N-1)k}$$

N 次乘法

	复数乘法	复数加法
一个 $X(k)$	N	$N-1$
N 个 $X(k)$	N^2	$N(N-1)$

```
function X = mydft(x)
    NF = length(x);
    X = zeros(NF,1);
    for k=0:NF-1
        for n=0:NF-1
            X(k+1) = X(k+1)+x(n+1)*exp(-j*2*pi*k*n/NF);
        endfor
    endfor
endfunction
```

直接计算DFT，运算量与 N 的平方成正比！

DFT问题与特点

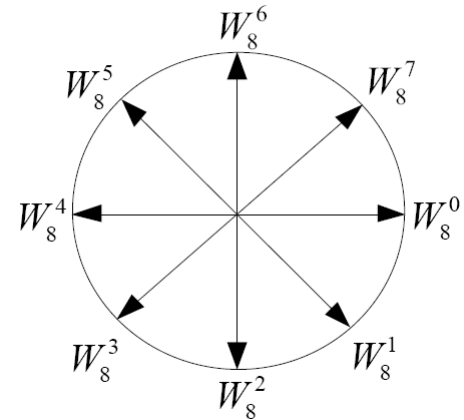
6

➤DFT的计算量分析

序列 长度	DFT	
	复数乘法 N^2	复数加法 $N(N-1)$
2	4	2
8	64	56
32	1024	992
64	4096	4032
256	65536	65280
1024	1048576	1047552
2048	4194304	4192256
2^{20}	$\approx 10^{12}$	$\approx 10^{12}$

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n)W_N^{nk}$$

$$W_N := e^{-j\frac{2\pi}{N}}, \quad W_N^{nk} := e^{-j\frac{2\pi}{N}nk}$$



➤ W矩阵的性质

$$W_N := e^{-j\frac{2\pi}{N}}, \quad W_N^{nk} := e^{-j\frac{2\pi}{N}nk}$$

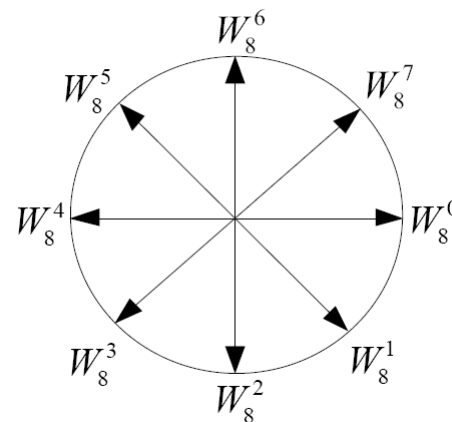
一点小Trick

$$\begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} = \begin{bmatrix} W^0 & W^0 & W^0 & \dots & W^0 \\ W^0 & W^{1 \times 1} & W^{2 \times 1} & \dots & W^{(N-1) \times 1} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ W^0 & W^{1 \times (N-1)} & W^{2 \times (N-1)} & \dots & W^{(N-1) \times (N-1)} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}$$

$$W_N = e^{-j\frac{2\pi}{N}}$$

$$W_N^0 = W_N^{kN} = 1, \quad W_N^{N/2} = -1$$

$$W_N^{N/4} = -j, \quad W_N^{3N/4} = j$$



➤ W 矩阵的性质

$$X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk}$$

$$k=0: X(0) = \sum_{n=0}^3 x(n) \cdot W_4^{nk} = x(0) \cdot W_4^0 + x(1) \cdot W_4^0 + x(2) \cdot W_4^0 + x(3) \cdot W_4^0$$

$$k=1: X(1) = \sum_{n=0}^3 x(n) \cdot W_4^{nk} = x(0) \cdot W_4^0 + x(1) \cdot W_4^1 + x(2) \cdot W_4^2 + x(3) \cdot W_4^3$$

$$k=2: X(2) = \sum_{n=0}^3 x(n) \cdot W_4^{nk} = x(0) \cdot W_4^0 + x(1) \cdot W_4^2 + x(2) \cdot W_4^4 + x(3) \cdot W_4^6$$

$$k=3: X(3) = \sum_{n=0}^3 x(n) \cdot W_4^{nk} = x(0) \cdot W_4^0 + x(1) \cdot W_4^3 + x(2) \cdot W_4^6 + x(3) \cdot W_4^9$$

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

$$W_N = e^{-j\frac{2\pi}{N}}$$

➤ W 矩阵的性质——周期性

$$W_N^{n(N-k)} = W_N^{-nk}, \quad W_N^{k(N-n)} = W_N^{-nk}$$

$$\begin{aligned} W_N^{n(N-k)} &= e^{-j\frac{2\pi}{N}n(N-k)} = e^{-j\frac{2\pi}{N}nN} \cdot e^{j\frac{2\pi}{N}nk} = e^{-j2\pi n} \cdot W_N^{-nk} = 1 \cdot W_N^{-nk} = W_N^{-nk} \\ W_N^{k(N-n)} &= e^{-j\frac{2\pi}{N}k(N-n)} = e^{-j\frac{2\pi}{N}kN} \cdot e^{j\frac{2\pi}{N}kn} = e^{-j2\pi k} \cdot W_N^{-nk} = 1 \cdot W_N^{-nk} = W_N^{-nk} \end{aligned}$$

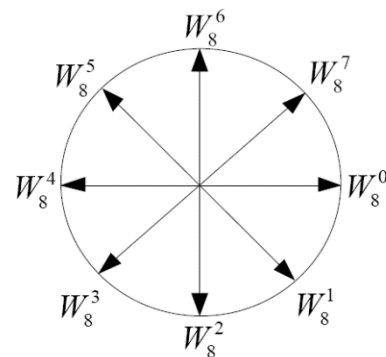
推论： $W_N^{nk} = W_N^{nk \% N}$

设：

$$nk = N \cdot q + r, \quad q \in \mathbb{Z}$$

进而有：

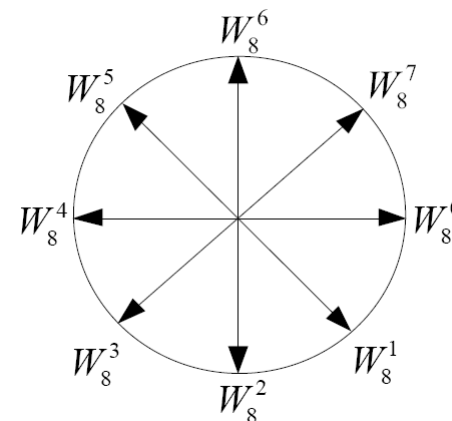
$$W_N^r = W_N^{(nk - N \cdot q)} = W_N^{nk} \cdot W_N^{-N \cdot q} = W_N^{nk} \cdot e^{j\frac{2\pi}{N} \cdot (-N \cdot q)} = W_N^{nk} \cdot 1 = W_N^{nk}$$



➤ W 矩阵的性质——对称性

$$W_N^{nk + \frac{N}{2}} = -W_N^{nk}$$

$$W_N^{nk + \frac{N}{2}} = W_N^{nk} W_N^{\frac{N}{2}} = W_N^{nk} e^{-j\pi}$$

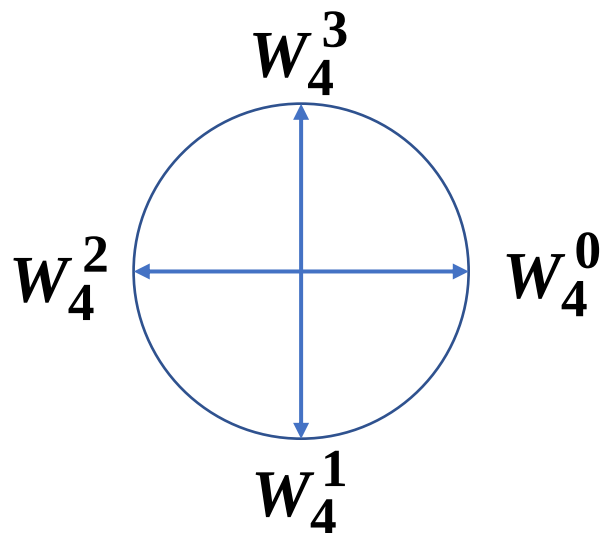
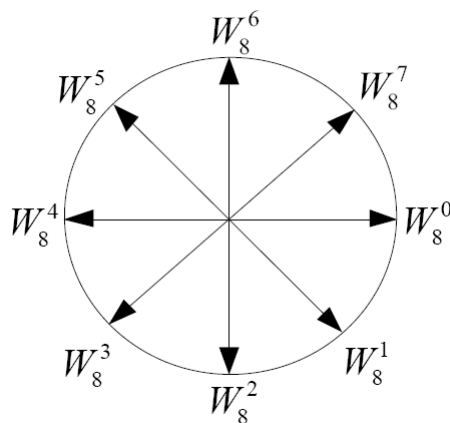


➤ W 矩阵的性质——缩放性

$$W_N^{2r} = W_{N/2}^r$$

$$W_N^{mr} = W_{N/m}^r$$

$$W_N^{2r} = e^{-j\frac{2\pi}{N}\cdot 2r} = e^{-j\frac{2\pi}{N/2}r} = W_{N/2}^r$$



$$W_N^2 = W_{N/2}$$

DFT问题与特点

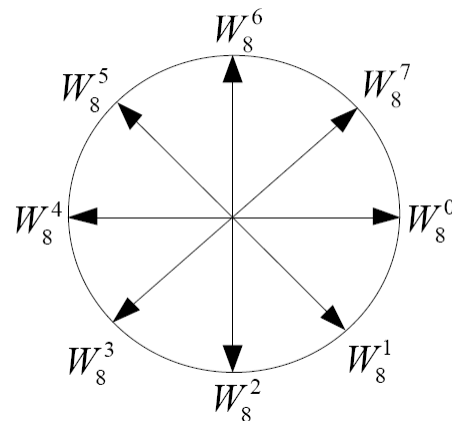
13

➤ W矩阵的性质——总结

周期性 $W_N^{nk+N} = W_N^{nk}$

对称性 $W_N^{nk+\frac{N}{2}} = -W_N^{nk}$

缩放性 $W_N^{2r} = W_{N/2}^r$



转一圈取正，转半圈取负

$$W_N = e^{-j\frac{2\pi}{N}}$$

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

Diagram illustrating the butterfly network for a 4-point DFT. Red arrows show the connections between input samples $x(n)$ and the corresponding twiddle factors W_4^k in the matrix. The matrix is structured to show the butterfly pattern, with the last element being W_4^9 (which is W_4^1).

➤ W矩阵的性质

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} W_4^0 & W_4^0 & W_4^0 & W_4^0 \\ W_4^0 & W_4^1 & W_4^2 & W_4^3 \\ W_4^0 & W_4^2 & W_4^4 & W_4^6 \\ W_4^0 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

$$W_N^{nk+N} = W_N^{nk}$$

$$W_N^{nk+\frac{N}{2}} = -W_N^{nk}$$

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & W_4^2 & W_4^3 \\ 1 & W_4^2 & 1 & W_4^6 \\ 1 & W_4^3 & W_4^6 & W_4^9 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix}$$

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$



规律

➤ W矩阵的性质

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

- $4 \cdot (4-1) = 12$ 次加法
- $4 \cdot 4 = 16$ 次乘法

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & W_4^1 & -W_4^1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -W_4^1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(2) \\ x(1) \\ x(3) \end{bmatrix}$$

分组

- $x(0), x(2)$
- $x(1), x(3)$

➤ W矩阵的性质

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

- $4 \cdot (4-1) = 12$ 次加法
- $4 \cdot 4 = 16$ 次乘法

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & W_4^1 & -W_4^1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -W_4^1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(2) \\ x(1) \\ x(3) \end{bmatrix}$$

分组

- $x(0), x(2)$
- $x(1), x(3)$

$$x(0) + x(2) := G(0)$$

$$k=0: X(0) = G(0) + H(0)$$

$$x(1) + x(3) := H(0)$$

$$k=1: X(1) = G(1) + W_4^1 H(1)$$

$$x(0) - x(2) := G(1)$$

$$k=2: X(2) = G(0) - H(0)$$

$$x(1) - x(3) := H(1)$$

$$k=3: X(3) = G(1) - W_4^1 H(1)$$

- $4+4=8$ 次加法
- 4次乘法

4次加法

4次加法, 4次乘法

➤ W矩阵的性质

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix} \quad \Rightarrow \quad \begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & W_4^1 & -W_4^1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -W_4^1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(2) \\ x(1) \\ x(3) \end{bmatrix} \quad \Rightarrow$$
$$\begin{aligned} k=0 : X(0) &= G(0) + H(0) \\ k=1 : X(1) &= G(1) + W_4^1 H(1) \\ k=2 : X(2) &= G(0) - H(0) \\ k=3 : X(3) &= G(1) - W_4^1 H(1) \end{aligned}$$

- 经过周期性与对称性简化之后，容易发现DFT运算中存在着不必要的重复计算，避免这种重复，是简化运算的关键。
- N点DFT运算可以分解为两组N/2点DFT运算，然后再取和。
- DFT的复杂度与点数N有关！

快速傅里叶变换

Principal Discoveries of Efficient Methods of Computing the DFT

Researcher(s)	Date	Sequence Lengths	Number of DFT Values	Application
C. F. Gauss [10]	1805	Any composite integer	All	Interpolation of orbits of celestial bodies
F. Carlini [28]	1828	12	—	Harmonic analysis of barometric pressure
A. Smith [25]	1846	4, 8, 16, 32	5 or 9	Correcting deviations in compasses on ships
J. D. Everett [23]	1860	12	5	Modeling underground temperature deviations
C. Runge [7]	1903	$2^n k$	All	Harmonic analysis of functions
K. Stumpff [16]	1939	$2^n k, 3^n k$	All	Harmonic analysis of functions
Danielson and Lanczos [5]	1942	2^n	All	X-ray diffraction in crystals
L. H. Thomas [13]	1948	Any integer with relatively prime factors	All	Harmonic analysis of functions
I. J. Good [3]	1958	Any integer with relatively prime factors	All	Harmonic analysis of functions
Cooley and Tukey [1]	1965	Any composite integer	All	Harmonic analysis of functions
S. Winograd [14]	1976	Any integer with relatively prime factors	All	Use of complexity theory for harmonic analysis

Principal Discoveries of Efficient Methods of Computing the DFT

Researcher(s)	Date	Sequence Lengths	Number of DFT Values	Application
C. F. Gauss [10]				高斯：计算小行星智神星 (Pallas) 和婚神星 (Juno) 的运行轨道
F. Carlini [28]				
A. Smith [25]	1846	4, 8, 16, 32	5 or 9	barometric pressure Correcting deviations in compasses on ships
J. D. Everett [23]				龙格：数值分析，将经验函数分解为正弦波
C. Runge [7]	1905	2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144, 524288, 1048576, 2097152, 4194304, 8388608, 16777216, 33554432, 67108864, 134217728, 268435456, 536870912, 1073741824, 2147483648, 4294967296, 8589934592, 17179869184, 34359738368, 68719476736, 137438953472, 274877906944, 549755813888, 1099511627776, 2199023255552, 4398046511104, 8796093022208, 17592186044416, 35184372088832, 70368744177664, 140737488355328, 281474976710656, 562949953421312, 1125899906842624, 2251799813685248, 4503599627370496, 9007199254740992, 18014398509481984, 36028797018963968, 72057594037927936, 144115188075855872, 288230376151711744, 576460752303423488, 1152921504606846976, 2305843009213693952, 4611686018427387904, 9223372036854775808, 18446744073709551616, 36893488147419103232, 73786976294838206464, 147573952589676412928, 295147905179352825856, 590295810358705651712, 1180591620717411303424, 2361183241434822606848, 4722366482869645213696, 9444732965739290427392, 18889465931478580854784, 37778931862957161709568, 75557863725914323419136, 151115727451828646838272, 302231454903657293676544, 604462909807314587353088, 1208925819614629174706176, 2417851639229258349412352, 4835703278458516698824704, 9671406556917033397649408, 19342813113834066795298816, 38685626227668133590597632, 77371252455336267181195264, 154742504910672534362390528, 309485009821345068724781056, 618970019642690137449562112, 1237940039285380274899124224, 2475880078570760549798248448, 4951760157141521099596496896, 9903520314283042199192993792, 19807040628566084398385987584, 39614081257132168796771975168, 79228162514264337593543950336, 158456325028528675187087900672, 316912650057057350374175801344, 633825300114114700748351602688, 1267650600228229401496703205376, 2535301200456458802993406410752, 5070602400912917605986812821504, 10141204801825835211973625643008, 20282409603651670423947251286016, 40564819207303340847894502572032, 81129638414606681695789005144064, 162259276829213363391578010288128, 324518553658426726783156020576256, 649037107316853453566312041152512, 1298074214633706907132624082305024, 2596148429267413814265248164610048, 5192296858534827628530496329220096, 10384593717069655257060992658440192, 20769187434139310514121985316880384, 41538374868278621028243970633760768, 83076749736557242056487941267521536, 166153499473114484112975882535043072, 332306998946228968225951765070086144, 664613997892457936451903530140172288, 1329227995784915872903807060280344576, 2658455991569831745807614120560689152, 5316911983139663491615228241121378304, 10633823966279326983230456482242756608, 21267647932558653966460912964485513216, 42535295865117307932921825928971026432, 85070591730234615865843651857942052864, 170141183460469231731687303715884105728, 340282366920938463463374607431768211456, 680564733841876926926749214863536422912, 1361129467683753853853498429727072845824, 2722258935367507707706996859454145691648, 5444517870735015415413993718908291383296, 10889035741470030830827987437816582766592, 21778071482940061661655974875633165533184, 43556142965880123323311949751266331066368, 87112285931760246646623899502532662132736, 174224571863520493293247799005065324265472, 348449143727040986586495598010130648530944, 696898287454081973172991196020261297061888, 1393796574908163946345982392040522594123776, 2787593149816327892691964784081045188247552, 5575186299632655785383929568162090376495104, 11150372599265311570767859136324180752990208, 22300745198530623141535718272648361505980416, 44601490397061246283071436545296723011960832, 89202980794122492566142873090593446023921664, 178405961588244985132285746181186892047843328, 356811923176489970264571492362373784095686656, 713623846352979940529142984724747568191373312, 1427247692705959881058285969449495136382746624, 2854495385411919762116571938898990272765493248, 5708990770823839524233143877797980545530986496, 11417981541647679048466287755595961091061972992, 22835963083295358096932575511191922182123945984, 45671926166590716193865151022383844364247891968, 91343852333181432387730302044767688728495783936, 182687704666362864775460604089535377456991567872, 365375409332725729550921208179070754913983135744, 730750818665451459101842416358141509827966271488, 1461501637330902918203684832716283019655932542976, 2923003274661805836407369665432566039311865085952, 5846006549323611672814739330865132078623730171904, 11692013098647223345629478661730264157247460343808, 23384026197294446691258957323460528314494920687616, 46768052394588893382517914646921056628989841375232, 93536104789177786765035829293842113257979682750464, 187072209578355573530071658587684226515959365500928, 374144419156711147060143317175368453031918731001856, 748288838313422294120286634350736906063837462003712, 1496577676626844588240573268701473812127674924007424, 2993155353253689176481146537402947624255349848014848, 5986310706507378352962293074805895248510699696029696, 11972621413014756705924586149611790497021399392059392, 23945242826029513411849172299223580994042798784118784, 47890485652059026823698344598447161988085597568237568, 95780971304118053647396689196894323976171195136475136, 191561942608236107294793378393788647952342390272950272, 383123885216472214589586756787577295904684780545900544, 766247770432944429179173513575154591809369561091801088, 1532495540865888858358347027150309183618739122183602176, 3064991081731777716716694054300618367237478244367204352, 6129982163463555433433388108601236734474956488734408704, 12259964326927110866866776217202473468949912977468817408, 24519928653854221733733552434404946937899825954937634816, 49039857307708443467467104868809893875799651909875269632, 98079714615416886934934209737619787751599303819750539264, 196159429230833773869868419475239575503198607639501078528, 392318858461667547739736838950479151006397215279002157056, 784637716923335095479473677900958302012794430558004314112, 1569275433846670190958947355801916604025588861116008628224, 3138550867693340381917894711603833208051177722232017256448, 6277101735386680763835789423207666416102355444464034512896, 12554203470773361527671578846415332832204710888928069025792, 25108406941546723055343157692830665664409421777856138051584, 50216813883093446110686315385661331328818843555712276103168, 100433627766186892221372630771322662657637687111424552206336, 200867255532373784442745261542645325315275374222849104412672, 401734511064747568885490523085290650630550748445698208825344, 803469022129495137770981046170581301261101496891396417650688, 1606938044258990275541962092341162602522202993782792835301376, 3213876088517980551083924184682325205044405987565585670602752, 6427752177035961102167848369364650410088811975131171341205504, 12855504354071922204335696738729300820177623950262342682411008, 25711008708143844408671393477458601640355247900524685364822016, 51422017416287688817342786954917203280710495801049370729644032, 102844034832575377634685573909834406561420991602098741459288064, 205688069665150755269371147819668813122841983204197482918576128, 411376139330301510538742295639337626245683966408394965837152256, 822752278660603021077484591278675252491367932816789931674304512, 1645504557321206042154969182557350504982735865633579863348609024, 3291009114642412084309938365114701009965471731267159726697218048, 6582018229284824168619876730229402019930943462534319453394436096, 13164036458569648337239753460458804039861886925068638906788872192, 26328072917139296674479506920917608079723773850137277813577744384, 52656145834278593348959013841835216159447547700274555627155488768, 105312291668557186697918027683670432318895095400549111254310977536, 210624583337114373395836055367340864637790190801098222508621955072, 421249166674228746791672110734681729275580381602196445017243910144, 842498333348457493583344221469363458551160763204392890034487820288, 1684996666696914987166688442938726917102321526408785780068975640576, 3369993333393829974333376885877453834204643052817571560137951281152, 6739986666787659948666753771754907668409286105635143120275902562304, 13479973333575319897333507543509815336818572211270286240551805124608, 26959946667150639794667015087019630673637144422540572481103610249216, 53919893334301279589334030174039261347274288845081144962207220498432, 107839786668602559178668060348078522694548577690162289924414440996864, 215679573337205118357336120696157045389097155380324579848828881993728, 431359146674410236714672241392314090778194310760649159697657763987456, 862718293348820473429344482784628181556388621521298319395315527974912, 1725436586697640946858688965569256363112777243042596638790631055949824, 3450873173395281893717377931138512726225554486085193277581262111899648, 6901746346790563787434755862277025452451108972170386555162524223799296, 13803492693581127574869511724554050904902217944340773110325048447598592, 27606985387162255149739023449108101809804435888681546220650096895197184, 55213970774324510299478046898216203619608871777363092441300193790394368, 110427941548649020598956093796432407239217743554726184882600387580788736, 220855883097298041197912187592864814478435487109452369765200775161577472, 441711766194596082395824375185729628956870974218904739530401550323154944, 883423532389192164791648750371459257913741948437809479060803100646309888, 1766847064778384329583297500742918515827483896875618958121606201292619776, 3533694129556768659166595001485837031654967793751237916243212402585239552, 7067388259113537318333190002971674063309935587502475832486424805170479104, 14134776518227074636666380005943348126619871175004951664972849610340958208, 28269553036454149273332760011886696253239742350009903329945699220681916416, 56539106072908298546665520023773392506479484700019806659891398441363832832, 113078212145816597093331040047546785012958969400039613319782796882727665664, 226156424291633194186662080095093570025917938800079226639565593765455331328, 452312848583266388373324160190187140051835877600158453279131187530910662656, 904625697166532776746648320380374280103671755200316906558262375061821325312, 1809251394333065553493296640760748560207343510400633813116524750123642650624, 3618502788666131106986593281521497120414687020801267626233049500247285301248, 7237005577332262213973186563042994240829374041602535252466099000494570602496, 14474011154664524427946373126085988481658748083205070504932198000989141204992, 28948022309329048855892746252171976963317496166410141009864396001978282409984, 57896044618658097711785492504343953926634992332820282019728792003956564819968, 115792089237316195423570985008687907853269984665640564039457584007913129639936, 231584178474632390847141970017375815706539969331281128078915168015826259279872, 463168356949264781694283940034751631413079938662562256157830336031652518559744, 926336713898529563388567880069503262826159877325124512315660672063305037119488, 1852673427797059126777135760139006525652319754650249024631321344126610074238976, 3705346855594118253554271520278013051304639509300498049262642688253220148477952, 7410693711188236507108543040556026102609279018600996098525285376506440296955904, 14821387422376473014217086081112052205218558037201992197050570753012880593911808, 29642774844752946028434172162224104410437116074403984394101141506025761187823616, 59285549689505892056868344324448208820874232148807968788202283012051522375647232, 118571099379011784113736688648896417641748464297615937576404566024103044751294464, 237142198758023568227473377297792835283496928595231875152809132048206089502588928, 474284397516047136454946754595585670566993857190463750305618264096412179005177856, 948568795032094272909893509191171341133987714380927500611236528192824358010355712, 1897137590064188545819787018382342682267975428761855001222473056385648716020711424, 3794275180128377091639574036764685364535950857523710002444946112771297432041422848, 7588550360256754183279148073529370729071901715047420004889892225542594864082845696, 15177100720513508366558296147058741458143803430094840009779784451085189728165691392, 303542014410270167331165922		

快速傅立叶变换

21

An Algorithm for the Machine Calculation of Complex Fourier Series

By James W. Cooley and John W. Tukey

An efficient method for the calculation of the interactions of a 2^m factorial experiment was introduced by Yates and is widely known by his name. The generalization to 3^m was given by Box et al. [1]. Good [2] generalized these methods and gave elegant algorithms for which one class of applications is the calculation of Fourier series. In their full generality, Good's methods are applicable to certain problems in which one must multiply an N -vector by an $N \times N$ matrix which can be factored into m sparse matrices, where m is proportional to $\log N$. This results in a procedure requiring a number of operations proportional to $N \log N$ rather than N^2 . These methods are applied here to the calculation of complex Fourier series. They are useful in situations where the number of data points is, or can be chosen to be, a highly composite number. The algorithm is here derived and presented in a rather different form. Attention is given to the choice of N . It is also shown how special advantage can be obtained in the use of a binary computer with $N = 2^m$ and how the entire calculation can be performed within the array of N data storage locations used for the given Fourier coefficients.

Consider the problem of calculating the complex Fourier series

$$(1) \quad X(j) = \sum_{k=0}^{N-1} A(k) \cdot W^{jk}, \quad j = 0, 1, \dots, N-1,$$

where the given Fourier coefficients $A(k)$ are complex and W is the principal N th root of unity,

$$(2) \quad W = e^{2\pi i/N}.$$

A straightforward calculation using (1) would require N^2 operations where "operation" means, as it will throughout this note, a complex multiplication followed by a complex addition.

The algorithm described here iterates on the array of given complex Fourier amplitudes and yields the result in less than $2N \log_2 N$ operations without requiring more data storage than is required for the given array A . To derive the algorithm, suppose N is a composite, i.e., $N = r_1 \cdot r_2$. Then let the indices in (1) be expressed

$$(3) \quad \begin{aligned} j &= j_1 r_1 + j_0, & j_0 &= 0, 1, \dots, r_1 - 1, & j_1 &= 0, 1, \dots, r_2 - 1, \\ k &= k_1 r_2 + k_0, & k_0 &= 0, 1, \dots, r_2 - 1, & k_1 &= 0, 1, \dots, r_1 - 1. \end{aligned}$$

Then, one can write

$$(4) \quad X(j_1, j_0) = \sum_{k_0} \sum_{k_1} A(k_1, k_0) \cdot W^{j_1 r_1 k_1 + j_0 k_0}.$$

Received August 17, 1964. Research in part at Princeton University under the sponsorship of the Army Research Office (Durham). The authors wish to thank Richard Garwin for his essential role in communication and encouragement.

MACHINE CALCULATION OF COMPLEX FOURIER SERIES 297

where values are as follows:

r	r_1	r_2
2	2	2
3	3	3
4	4	4
5	5	5
6	6	6
7	7	7
8	8	8
9	9	9
10	10	10

The use of $r_1 = 2$ is usually most efficient, but the gain is only about 5% over the use of 3 or 4, which have other advantages. If necessary, the use of r_1 up to 10 can increase the number of operations by no more than 50%. Accordingly, we can find "highly composite" values of N within a few percent of any given large number.

Whenever possible, the use of $N = 2^m$ with $r = 2$ offers important advantages for computers with binary arithmetic, both in addressing and in multiplication economy.

The algorithm with $r = 2$ is derived by expressing the indices in the form

$$(11) \quad j = j_m 2^{m-1} + \dots + j_1 2 + j_0,$$

where j_m and j_0 are equal to 0 or 1 and are the contents of the respective bit positions in the binary representation of j . All arrays will now be written as functions of the bits of their indices. With this convention, (11) is written

$$(12) \quad X(j_m, \dots, j_0) = \sum_{k_m, \dots, k_0} A(k_m, \dots, k_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0},$$

where the sums are over $k_r = 0, 1$. Since

$$(13) \quad A(k_m, \dots, k_0) = \sum_{j_m, \dots, j_0} A(j_m, \dots, j_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0},$$

preceding to the next summation sum, over $k_m, \dots, k_1$, and so on, and using

$$(14) \quad A(k_m, \dots, k_0) = \sum_{j_m, \dots, j_0} A(j_m, \dots, j_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0},$$

one obtains successive arrays,

$$(15) \quad A(k_m, \dots, k_0) = \sum_{j_m, \dots, j_0} A(j_m, \dots, j_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0},$$

for $r = 1, 2, \dots, m$.

JAMES W. COOLEY AND JOHN W. TUKEY

298

Since

$$(16) \quad W^{j_m 2^{m-1} k_m} = W^{j_m 2^{m-1} k_m},$$

the inner sum, over k_m , depends only on j_m and k_m and can be defined as a new array,

$$(17) \quad A(k_m, k_0) = \sum_{j_m, \dots, j_0} A(j_m, \dots, j_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0}.$$

The result can then be written

$$(18) \quad X(j_m, \dots, j_0) = \sum_{k_m, \dots, k_0} A(k_m, k_0) W^{j_m 2^{m-1} k_m + \dots + j_1 2 k_1 + j_0 k_0}.$$

There are N elements in the array A , each requiring r operations, giving a total of Nr operations to obtain A . Similarly, it takes r operations to calculate X from A . Therefore, the two-way algorithm, given by (15) and (17), requires a total

$$(19) \quad T = N(r_1 + r_2)$$

operations.

It may be seen how successive applications of the above procedure, starting with its application to (5), give us a way algorithm requiring

$$(20) \quad T = N(r_1 + r_2 + \dots + r_m)$$

operations, where

$$(21) \quad N = N_1 \cdot N_2 \cdot \dots \cdot N_m,$$

$N_1 = 2^m$ with $r_1 = 2$, $N_2 = 2^{m-1}$ with $r_2 = 2$, $N_3 = 2^{m-2}$ with $r_3 = 2$, $N_4 = 2^{m-3}$ with $r_4 = 2$, $N_5 = 2^{m-4}$ with $r_5 = 2$, $N_6 = 2^{m-5}$ with $r_6 = 2$, $N_7 = 2^{m-6}$ with $r_7 = 2$, $N_8 = 2^{m-7}$ with $r_8 = 2$, $N_9 = 2^{m-8}$ with $r_9 = 2$, $N_{10} = 2^{m-9}$ with $r_{10} = 2$, $N_{11} = 2^{m-10}$ with $r_{11} = 2$, $N_{12} = 2^{m-11}$ with $r_{12} = 2$, $N_{13} = 2^{m-12}$ with $r_{13} = 2$, $N_{14} = 2^{m-13}$ with $r_{14} = 2$, $N_{15} = 2^{m-14}$ with $r_{15} = 2$, $N_{16} = 2^{m-15}$ with $r_{16} = 2$, $N_{17} = 2^{m-16}$ with $r_{17} = 2$, $N_{18} = 2^{m-17}$ with $r_{18} = 2$, $N_{19} = 2^{m-18}$ with $r_{19} = 2$, $N_{20} = 2^{m-19}$ with $r_{20} = 2$, $N_{21} = 2^{m-20}$ with $r_{21} = 2$, $N_{22} = 2^{m-21}$ with $r_{22} = 2$, $N_{23} = 2^{m-22}$ with $r_{23} = 2$, $N_{24} = 2^{m-23}$ with $r_{24} = 2$, $N_{25} = 2^{m-24}$ with $r_{25} = 2$, $N_{26} = 2^{m-25}$ with $r_{26} = 2$, $N_{27} = 2^{m-26}$ with $r_{27} = 2$, $N_{28} = 2^{m-27}$ with $r_{28} = 2$, $N_{29} = 2^{m-28}$ with $r_{29} = 2$, $N_{30} = 2^{m-29}$ with $r_{30} = 2$, $N_{31} = 2^{m-30}$ with $r_{31} = 2$, $N_{32} = 2^{m-31}$ with $r_{32} = 2$, $N_{33} = 2^{m-32}$ with $r_{33} = 2$, $N_{34} = 2^{m-33}$ with $r_{34} = 2$, $N_{35} = 2^{m-34}$ with $r_{35} = 2$, $N_{36} = 2^{m-35}$ with $r_{36} = 2$, $N_{37} = 2^{m-36}$ with $r_{37} = 2$, $N_{38} = 2^{m-37}$ with $r_{38} = 2$, $N_{39} = 2^{m-38}$ with $r_{39} = 2$, $N_{40} = 2^{m-39}$ with $r_{40} = 2$, $N_{41} = 2^{m-40}$ with $r_{41} = 2$, $N_{42} = 2^{m-41}$ with $r_{42} = 2$, $N_{43} = 2^{m-42}$ with $r_{43} = 2$, $N_{44} = 2^{m-43}$ with $r_{44} = 2$, $N_{45} = 2^{m-44}$ with $r_{45} = 2$, $N_{46} = 2^{m-45}$ with $r_{46} = 2$, $N_{47} = 2^{m-46}$ with $r_{47} = 2$, $N_{48} = 2^{m-47}$ with $r_{48} = 2$, $N_{49} = 2^{m-48}$ with $r_{49} = 2$, $N_{50} = 2^{m-49}$ with $r_{50} = 2$, $N_{51} = 2^{m-50}$ with $r_{51} = 2$, $N_{52} = 2^{m-51}$ with $r_{52} = 2$, $N_{53} = 2^{m-52}$ with $r_{53} = 2$, $N_{54} = 2^{m-53}$ with $r_{54} = 2$, $N_{55} = 2^{m-54}$ with $r_{55} = 2$, $N_{56} = 2^{m-55}$ with $r_{56} = 2$, $N_{57} = 2^{m-56}$ with $r_{57} = 2$, $N_{58} = 2^{m-57}$ with $r_{58} = 2$, $N_{59} = 2^{m-58}$ with $r_{59} = 2$, $N_{60} = 2^{m-59}$ with $r_{60} = 2$, $N_{61} = 2^{m-60}$ with $r_{61} = 2$, $N_{62} = 2^{m-61}$ with $r_{62} = 2$, $N_{63} = 2^{m-62}$ with $r_{63} = 2$, $N_{64} = 2^{m-63}$ with $r_{64} = 2$, $N_{65} = 2^{m-64}$ with $r_{65} = 2$, $N_{66} = 2^{m-65}$ with $r_{66} = 2$, $N_{67} = 2^{m-66}$ with $r_{67} = 2$, $N_{68} = 2^{m-67}$ with $r_{68} = 2$, $N_{69} = 2^{m-68}$ with $r_{69} = 2$, $N_{70} = 2^{m-69}$ with $r_{70} = 2$, $N_{71} = 2^{m-70}$ with $r_{71} = 2$, $N_{72} = 2^{m-71}$ with $r_{72} = 2$, $N_{73} = 2^{m-72}$ with $r_{73} = 2$, $N_{74} = 2^{m-73}$ with $r_{74} = 2$, $N_{75} = 2^{m-74}$ with $r_{75} = 2$, $N_{76} = 2^{m-75}$ with $r_{76} = 2$, $N_{77} = 2^{m-76}$ with $r_{77} = 2$, $N_{78} = 2^{m-77}$ with $r_{78} = 2$, $N_{79} = 2^{m-78}$ with $r_{79} = 2$, $N_{80} = 2^{m-79}$ with $r_{80} = 2$, $N_{81} = 2^{m-80}$ with $r_{81} = 2$, $N_{82} = 2^{m-81}$ with $r_{82} = 2$, $N_{83} = 2^{m-82}$ with $r_{83} = 2$, $N_{84} = 2^{m-83}$ with $r_{84} = 2$, $N_{85} = 2^{m-84}$ with $r_{85} = 2$, $N_{86} = 2^{m-85}$ with $r_{86} = 2$, $N_{87} = 2^{m-86}$ with $r_{87} = 2$, $N_{88} = 2^{m-87}$ with $r_{88} = 2$, $N_{89} = 2^{m-88}$ with $r_{89} = 2$, $N_{90} = 2^{m-89}$ with $r_{90} = 2$, $N_{91} = 2^{m-90}$ with $r_{91} = 2$, $N_{92} = 2^{m-91}$ with $r_{92} = 2$, $N_{93} = 2^{m-92}$ with $r_{93} = 2$, $N_{94} = 2^{m-93}$ with $r_{94} = 2$, $N_{95} = 2^{m-94}$ with $r_{95} = 2$, $N_{96} = 2^{m-95}$ with $r_{96} = 2$, $N_{97} = 2^{m-96}$ with $r_{97} = 2$, $N_{98} = 2^{m-97}$ with $r_{98} = 2$, $N_{99} = 2^{m-98}$ with $r_{99} = 2$, $N_{100} = 2^{m-99}$ with $r_{100} = 2$, $N_{101} = 2^{m-100}$ with $r_{101} = 2$, $N_{102} = 2^{m-101}$ with $r_{102} = 2$, $N_{103} = 2^{m-102}$ with $r_{103} = 2$, $N_{104} = 2^{m-103}$ with $r_{104} = 2$, $N_{105} = 2^{m-104}$ with $r_{105} = 2$, $N_{106} = 2^{m-105}$ with $r_{106} = 2$, $N_{107} = 2^{m-106}$ with $r_{107} = 2$, $N_{108} = 2^{m-107}$ with $r_{108} = 2$, $N_{109} = 2^{m-108}$ with $r_{109} = 2$, $N_{110} = 2^{m-109}$ with $r_{110} = 2$, $N_{111} = 2^{m-110}$ with $r_{111} = 2$, $N_{112} = 2^{m-111}$ with $r_{112} = 2$, $N_{113} = 2^{m-112}$ with $r_{113} = 2$, $N_{114} = 2^{m-113}$ with $r_{114} = 2$, $N_{115} = 2^{m-114}$ with $r_{115} = 2$, $N_{116} = 2^{m-115}$ with $r_{116} = 2$, $N_{117} = 2^{m-116}$ with $r_{117} = 2$, $N_{118} = 2^{m-117}$ with $r_{118} = 2$, $N_{119} = 2^{m-118}$ with $r_{119} = 2$, $N_{120} = 2^{m-119}$ with $r_{120} = 2$, $N_{121} = 2^{m-120}$ with $r_{121} = 2$, $N_{122} = 2^{m-121}$ with $r_{122} = 2$, $N_{123} = 2^{m-122}$ with $r_{123} = 2$, $N_{124} = 2^{m-123}$ with $r_{124} = 2$, $N_{125} = 2^{m-124}$ with $r_{125} = 2$, $N_{126} = 2^{m-125}$ with $r_{126} = 2$, $N_{127} = 2^{m-126}$ with $r_{127} = 2$, $N_{128} = 2^{m-127}$ with $r_{128} = 2$, $N_{129} = 2^{m-128}$ with $r_{129} = 2$, $N_{130} = 2^{m-129}$ with $r_{130} = 2$, $N_{131} = 2^{m-130}$ with $r_{131} = 2$, $N_{132} = 2^{m-131}$ with $r_{132} = 2$, $N_{133} = 2^{m-132}$ with $r_{133} = 2$, $N_{134} = 2^{m-133}$ with $r_{134} = 2$, $N_{135} = 2^{m-134}$ with $r_{135} = 2$, $N_{136} = 2^{m-135}$ with $r_{136} = 2$, $N_{137} = 2^{m-136}$ with $r_{137} = 2$, $N_{138} = 2^{m-137}$ with $r_{138} = 2$, $N_{139} = 2^{m-138}$ with $r_{139} = 2$, $N_{140} = 2^{m-139}$ with $r_{140} = 2$, $N_{141} = 2^{m-140}$ with $r_{141} = 2$, $N_{142} = 2^{m-141}$ with $r_{142} = 2$, $N_{143} = 2^{m-142}$ with $r_{143} = 2$, $N_{144} = 2^{m-143}$ with $r_{144} = 2$, $N_{145} = 2^{m-144}$ with $r_{145} = 2$, $N_{146} = 2^{m-145}$ with $r_{146} = 2$, $N_{147} = 2^{m-146}$ with $r_{147} = 2$, $N_{148} = 2^{m-147}$ with $r_{148} = 2$, $N_{149} = 2^{m-148}$ with $r_{149} = 2$, $N_{150} = 2^{m-149}$ with $r_{150} = 2$, $N_{151} = 2^{m-150}$ with $r_{151} = 2$, $N_{152} = 2^{m-151}$ with $r_{152} = 2$, $N_{153} = 2^{m-152}$ with $r_{153} = 2$, $N_{154} = 2^{m-153}$ with $r_{154} = 2$, $N_{155} = 2^{m-154}$ with $r_{155} = 2$, $N_{156} = 2^{m-155}$ with $r_{156} = 2$, $N_{157} = 2^{m-156}$ with $r_{157} = 2$, $N_{158} = 2^{m-157}$ with $r_{158} = 2$, $N_{159} = 2^{m-158}$ with $r_{159} = 2$, $N_{160} = 2^{m-159}$ with $r_{160} = 2$, $N_{161} = 2^{m-160}$ with $r_{161} = 2$, $N_{162} = 2^{m-161}$ with $r_{162} = 2$, $N_{163} = 2^{m-162}$ with $r_{163} = 2$, $N_{164} = 2^{m-163}$ with $r_{164} = 2$, $N_{165} = 2^{m-164}$ with $r_{165} = 2$, $N_{166} = 2^{m-165}$ with $r_{166} = 2$, $N_{167} = 2^{m-166}$ with $r_{167} = 2$, $N_{168} = 2^{m-167}$ with $r_{168} = 2$, $N_{169} = 2^{m-168}$ with $r_{169} = 2$, $N_{170} = 2^{m-169}$ with $r_{170} = 2$, $N_{171} = 2^{m-170}$ with $r_{171} = 2$, $N_{172} = 2^{m-171}$ with $r_{172} = 2$, $N_{173} = 2^{m-172}$ with $r_{173} = 2$, $N_{174} = 2^{m-173}$ with $r_{174} = 2$, $N_{175} = 2^{m-174}$ with $r_{175} = 2$, $N_{176} = 2^{m-175}$ with $r_{176} = 2$, $N_{177} = 2^{m-176}$ with $r_{177} = 2$, $N_{178} = 2^{m-177}$ with $r_{178} = 2$, $N_{179} = 2^{m-178}$ with $r_{179} = 2$, $N_{180} = 2^{m-179}$ with $r_{180} = 2$, $N_{181} = 2^{m-180}$ with $r_{181} = 2$, $N_{182} = 2^{m-181}$ with $r_{182} = 2$, $N_{183} = 2^{m-182}$ with $r_{183} = 2$, $N_{184} = 2^{m-183}$ with $r_{184} = 2$, $N_{185} = 2^{m-184}$ with $r_{185} = 2$, $N_{186} = 2^{m-185}$ with $r_{186} = 2$, $N_{187} = 2^{m-186}$ with $r_{187} = 2$, $N_{188} = 2^{m-187}$ with $r_{188} = 2$, $N_{189} = 2^{m-188}$ with $r_{189} = 2$, $N_{190} = 2^{m-189}$ with $r_{190} = 2$, $N_{191} = 2^{m-190}$ with $r_{191} = 2$, $N_{192} = 2^{m-191}$ with $r_{192} = 2$, $N_{193} = 2^{m-192}$ with $r_{193} = 2$, $N_{194} = 2^{m-193}$ with $r_{194} = 2$, $N_{195} = 2^{m-194}$ with $r_{195} = 2$, $N_{196} = 2^{m-195}$ with $r_{196} = 2$, $N_{197} = 2^{m-196}$ with $r_{197} = 2$, $N_{198} = 2^{m-197}$ with $r_{198} = 2$, $N_{199} = 2^{m-198}$ with $r_{199} = 2$, $N_{200} = 2^{m-199}$ with $r_{200} = 2$, $N_{201} = 2^{m-200}$ with $r_{201} = 2$, $N_{202} = 2^{m-201}$ with $r_{202} = 2$, $N_{203} = 2^{m-202}$ with $r_{203} = 2$, $N_{204} = 2^{m-203}$ with $r_{204} = 2$, $N_{205} = 2^{m-204}$ with $r_{205} = 2$, $N_{206} = 2^{m-205}$ with $r_{206} = 2$, $N_{207} = 2^{m-206}$ with $r_{207} = 2$, $N_{208} = 2^{m-207}$ with $r_{208} = 2$, $N_{209} = 2^{m-208}$ with $r_{209} = 2$, $N_{210} = 2^{m-209}$ with $r_{210} = 2$, $N_{211} = 2^{m-210}$ with $r_{211} = 2$, $N_{212} = 2^{m-211}$ with $r_{212} = 2$, $N_{213} = 2^{m-212}$ with $r_{213} = 2$, $N_{214} = 2^{m-213}$ with $r_{214} = 2$, $N_{215} = 2^{m-214}$ with $r_{215} = 2$, $N_{216} = 2^{m-215}$ with $r_{216} = 2$, $N_{217} = 2^{m-216}$ with $r_{217} = 2$, $N_{218} = 2^{m-217}$ with $r_{218} = 2$, $N_{219} = 2^{m-218}$ with $r_{219} = 2$, $N_{220} = 2^{m-219}$ with $r_{220} = 2$, $N_{221} = 2^{m-220}$ with $r_{221} = 2$, $N_{222} = 2^{m-221}$ with $r_{222} = 2$, $N_{223} = 2^{m-222}$ with $r_{223} = 2$, $N_{224} = 2^{m-223}$ with $r_{224} = 2$, $N_{225} = 2^{m-224}$ with $r_{225} = 2$, $N_{226} = 2^{m-225}$ with $r_{226} = 2$, $N_{227} = 2^{m-226}$ with $r_{227} = 2$, $N_{228} = 2^{m-227}$ with $r_{228} = 2$, $N_{229} = 2^{m-228}$ with $r_{229} = 2$, $N_{230} = 2^{m-229}$ with $r_{230} = 2$, $N_{231} = 2^{m-230}$ with $r_{231} = 2$, $N_{232} = 2^{m-231}$ with $r_{232} = 2$, $N_{233} = 2^{m-232}$ with $r_{233} = 2$, $N_{234} = 2^{m-233}$ with $r_{234} = 2$, $N_{235} = 2^{m-234}$ with $r_{235} = 2$, $N_{236} = 2^{m-235}$ with $r_{236} = 2$, $N_{237} = 2^{m-236}$ with $r_{237} = 2$, $N_{238} = 2^{m-237}$ with $r_{238} = 2$, $N_{239} = 2^{m-238}$ with $r_{239} = 2$, $N_{240} = 2^{m-239}$ with $r_{240} = 2$, $N_{241} = 2^{m-240}$ with $r_{241} = 2$, $N_{242} = 2^{m-241}$ with $r_{242} = 2$, $N_{243} = 2^{m-242}$ with $r_{243} = 2$, $N_{244} = 2^{m-243}$ with $r_{244} = 2$, $N_{245} = 2^{m-244}$ with $r_{245} = 2$, $N_{246} = 2^{m-245}$ with $r_{246} = 2$, $N_{247} = 2^{m-246}$ with $r_{247} = 2$, $N_{248} = 2^{m-247}$ with $r_{248} = 2$, $N_{249} = 2^{m-248}$ with $r_{249} = 2$, $N_{250} = 2^{m-249}$ with $r_{250} = 2$, $N_{251} = 2^{m-250}$ with $r_{251} = 2$, $N_{252} = 2^{m-251}$ with $r_{252} = 2$, $N_{253} = 2^{m-252}$ with $r_{253} = 2$, $N_{254} = 2^{m-253}$ with $r_{254} = 2$, $N_{255} = 2^{m-254}$ with $r_{255} = 2$, $N_{256} = 2^{m-255}$ with $r_{256} = 2$,

Stigler's Law: No scientific discovery is named after its original discoverer.

反复发明反复遗忘

John Tukey



James Cooley



➤ 分治法

利用分治法降低计算量

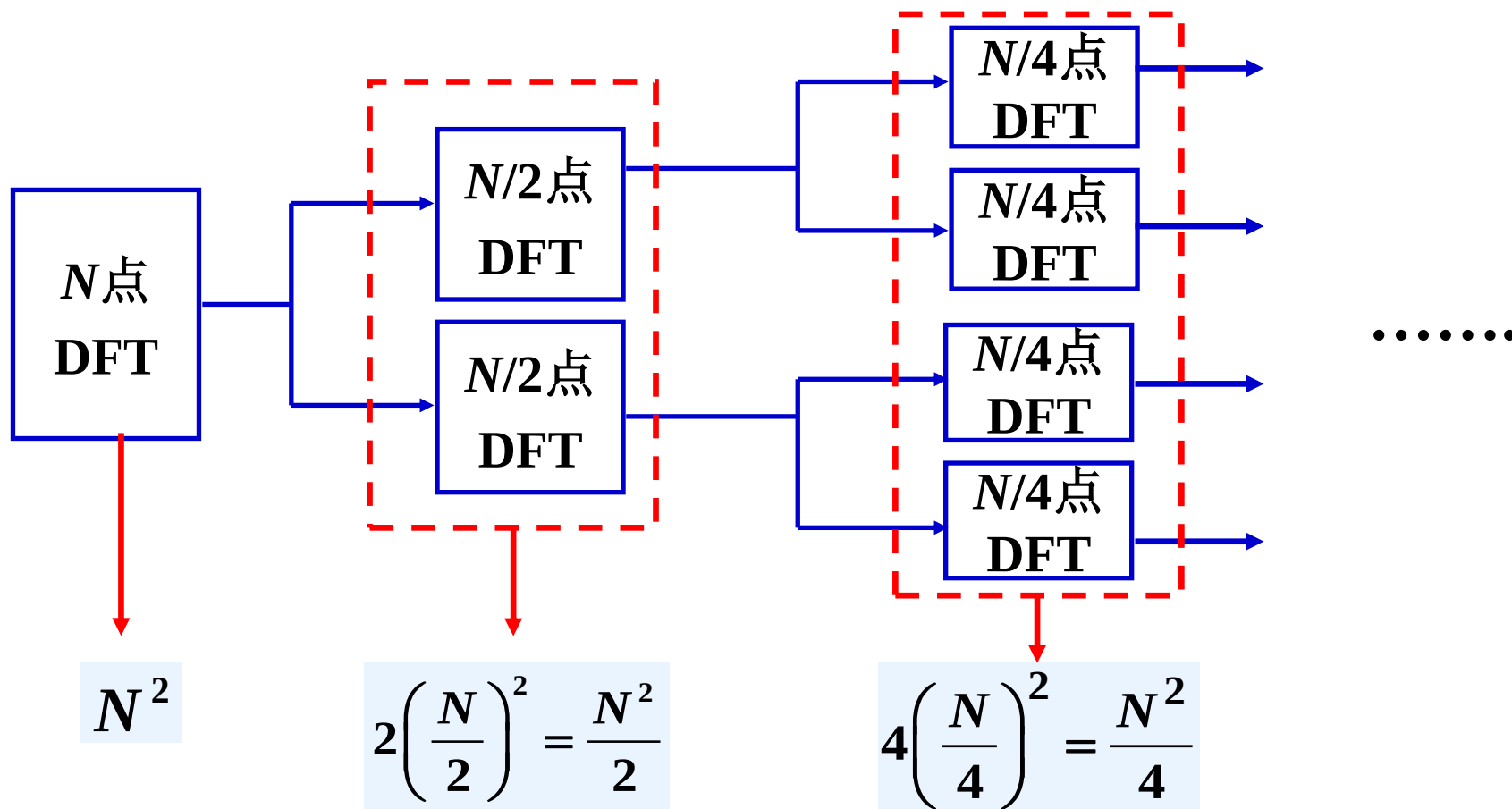
- **分治 (Divide and Conquer Algorithm)**
 - 把一个规模为 N 的问题分解为 K 个规模较小的问题，这些子问题相互独立，而且与原问题性质相同。求出子问题的解，就能得到原问题的解。
- **分治法解题的一般步骤：**
 - (1) 分解，将要解决的问题划分成若干规模较小的同类问题；
 - (2) 求解，当子问题划分得足够小时，用较简单的方法解决；
 - (3) 合并，按原问题的要求，将子问题的解逐层合并构成原问题的解。

DFT计算采用分治法的好处！



➤ 基本思想

将长序列的DFT分解为几个短序列的DFT



➤ 基本思想

DFT的点数 $N=2^M$ ，称为**基2FFT**。

长度为 $N=2^M$ 的序列，可以等分 $M-1$ 次后，最终成为 2^{M-1} ($N/2$) 个长度为 2 的序列。

两种基2FFT算法：

◆ 按时间抽取的FFT算法

DIT: Decimation-In-Time

◆ 按频率抽取的FFT算法

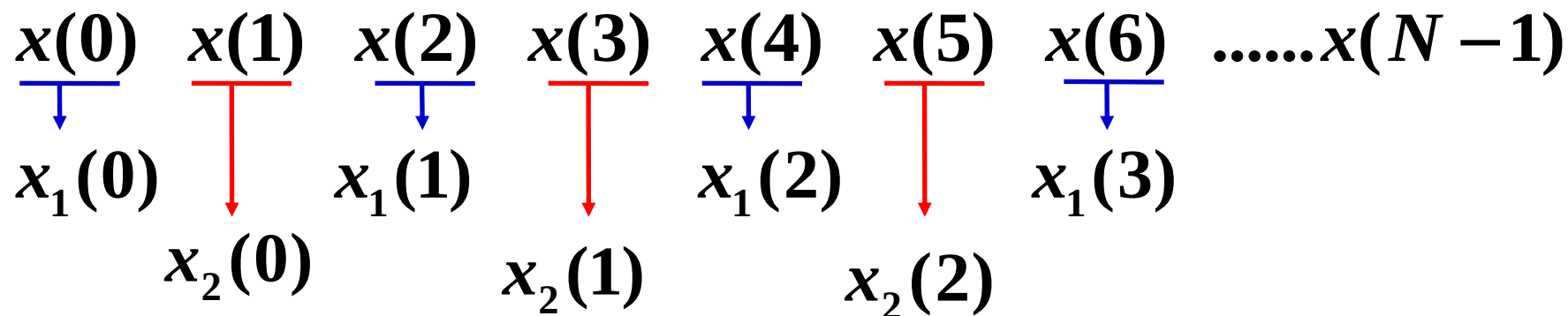
DIF: Decimation-In-Frequency

快速傅里叶变换

按时间抽取

➤ 基本原理

将序列 $x(n)$ 按 n 的奇偶分成两组：



一个 N 点长序列分为两个 $N/2$ 点长序列

$$x(2r) = x_1(r), \quad x(2r+1) = x_2(r) \quad r = 0, 1, \dots, N/2 - 1$$

➤ 基本思想

一个 N 点长序列分为两个 $N/2$ 点长序列

$$x(2r) = x_1(r), \quad x(2r+1) = x_2(r) \quad r = 0, 1, \dots, N/2 - 1$$

$$x(n) \xrightarrow{\text{N点DFT}} X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad k=0, 1, \dots, N-1$$

$$x_1(r) \xrightarrow{\text{N/2点DFT}} X_1(k) = \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk}$$

$$x_2(r) \xrightarrow{\text{N/2点DFT}} X_2(k) = \sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk} \quad k=0, 1, \dots, N/2-1$$

问题： 如何由 $X_1(k)$ 、 $X_2(k)$ 构造 $X(k)$ ？

FFT: 按时间抽取

29

➤ 基本思想

$$\begin{aligned} DFT[x(n)] = X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{\substack{n=0 \\ n \text{ 为偶数}}}^{N-1} x(n) W_N^{nk} + \sum_{\substack{n=0 \\ n \text{ 为奇数}}}^{N-1} x(n) W_N^{nk} \\ &\stackrel{\text{令 } n=2r}{=} \sum_{r=0}^{N/2-1} x(\mathbf{2r}) W_N^{\mathbf{2rk}} + \sum_{r=0}^{N/2-1} x(\mathbf{2r+1}) W_N^{(\mathbf{2r+1})k} \\ &\stackrel{\text{令 } n=2r+1}{=} \sum_{r=0}^{N/2-1} x_1(r) \left(W_N^2 \right)^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) \left(W_N^2 \right)^{rk} \end{aligned}$$

$x_1(r)$ 与 $x_2(r)$ 的长度为 $N/2$ ，但 W_N 考虑的是长度为 N

$$W_N^2 = e^{-j\frac{2\pi}{N} \cdot 2} = e^{-j\frac{2\pi}{N/2}} = W_{N/2}$$

➤ 基本思想

$$\begin{aligned}
 DFT[x(n)] = X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{\substack{n=0 \\ n \text{ 为偶数}}}^{N-1} x(n) W_N^{nk} + \sum_{\substack{n=0 \\ n \text{ 为奇数}}}^{N-1} x(n) W_N^{nk} \\
 &\quad \swarrow \text{令 } n=2r \quad \searrow \text{令 } n=2r+1 \\
 &= \sum_{r=0}^{N/2-1} x(2r) W_N^{2rk} + \sum_{r=0}^{N/2-1} x(2r+1) W_N^{(2r+1)k} \\
 &= \sum_{r=0}^{N/2-1} x_1(r) (W_N^2)^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) (W_N^2)^{rk}
 \end{aligned}$$

$x_1(r)$ 与 $x_2(r)$ 的长度为 $N/2$ ，但 W_N 考虑的是长度为 N

$$\begin{aligned}
 W_N^2 &= W_{N/2} \\
 &= \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk}
 \end{aligned}$$

➤ 基本思想

$$\begin{aligned} DFT[x(n)] &= X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \\ &= \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk} \\ &= X_1(k) + W_N^k X_2(k) \quad k = 0, 1, \dots, N/2 - 1 \end{aligned}$$



$$n = 0, 1, \dots, N-1 \quad k = 0, 1, \dots, N-1$$



$$r = 0, 1, \dots, (N/2 - 1) \quad k = 0, 1, \dots, (N/2 - 1)$$



只能得到一半的表示

FFT: 按时间抽取

32

➤ 基本思想

$$\begin{aligned} DFT[x(n)] = X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} \\ &= \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk} + W_N^k \sum_{r=0}^{N/2-1} x_2(r) W_{N/2}^{rk} \\ &= X_1(k) + W_N^k X_2(k) \end{aligned} \quad k = 0, 1, \dots, N/2-1$$

$$X(k + \frac{N}{2}) = X_1(k + \frac{N}{2}) + W_N^{k+N/2} X_2(k + \frac{N}{2})$$

$$X_1(k + \frac{N}{2}) = \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{r(k+N/2)} = \sum_{r=0}^{N/2-1} x_1(r) W_{N/2}^{rk} = X_1(k)$$

$$X_2(k + \frac{N}{2}) = X_2(k)$$

$$W_N^{k+N/2} = -W_N^k$$

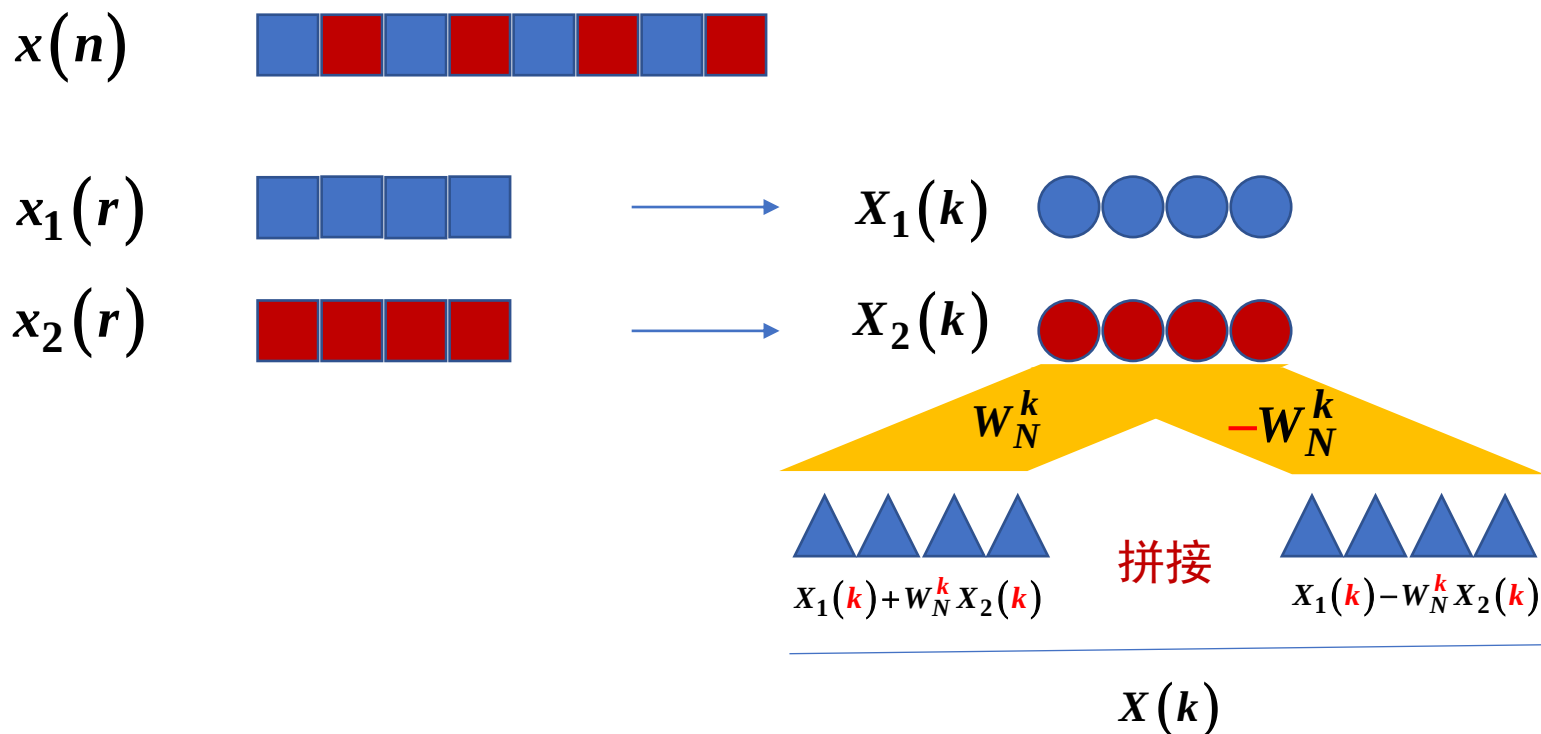
$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$

➤ 基本思想

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$

$$k = 0, 1, \dots, N/2 - 1$$

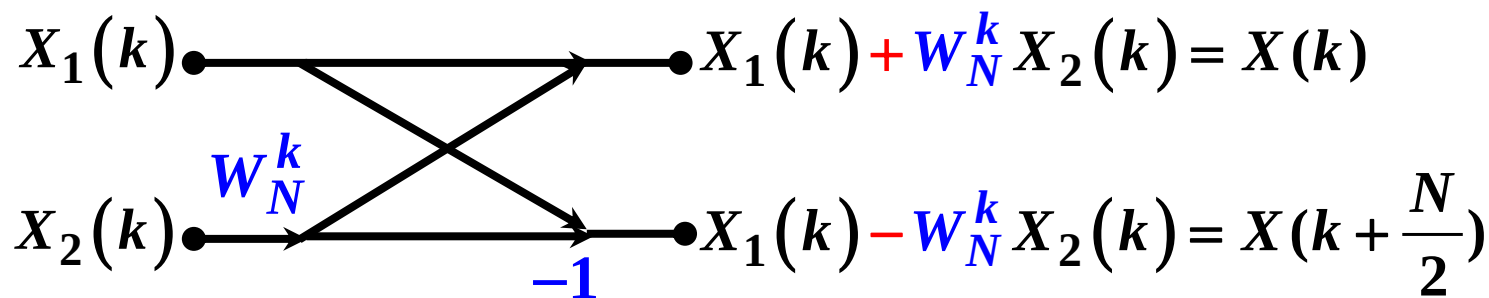


➤ 基本思想

由 $X_1(k)$ 、 $X_2(k)$ 求解 $X(k)$ 的公式：

$$X(k) = X_1(k) + W_N^k X_2(k) \quad k = 0, 1, \dots, N/2 - 1$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$



蝶形运算符号 (Butterfly Computation)

FFT: 按时间抽取

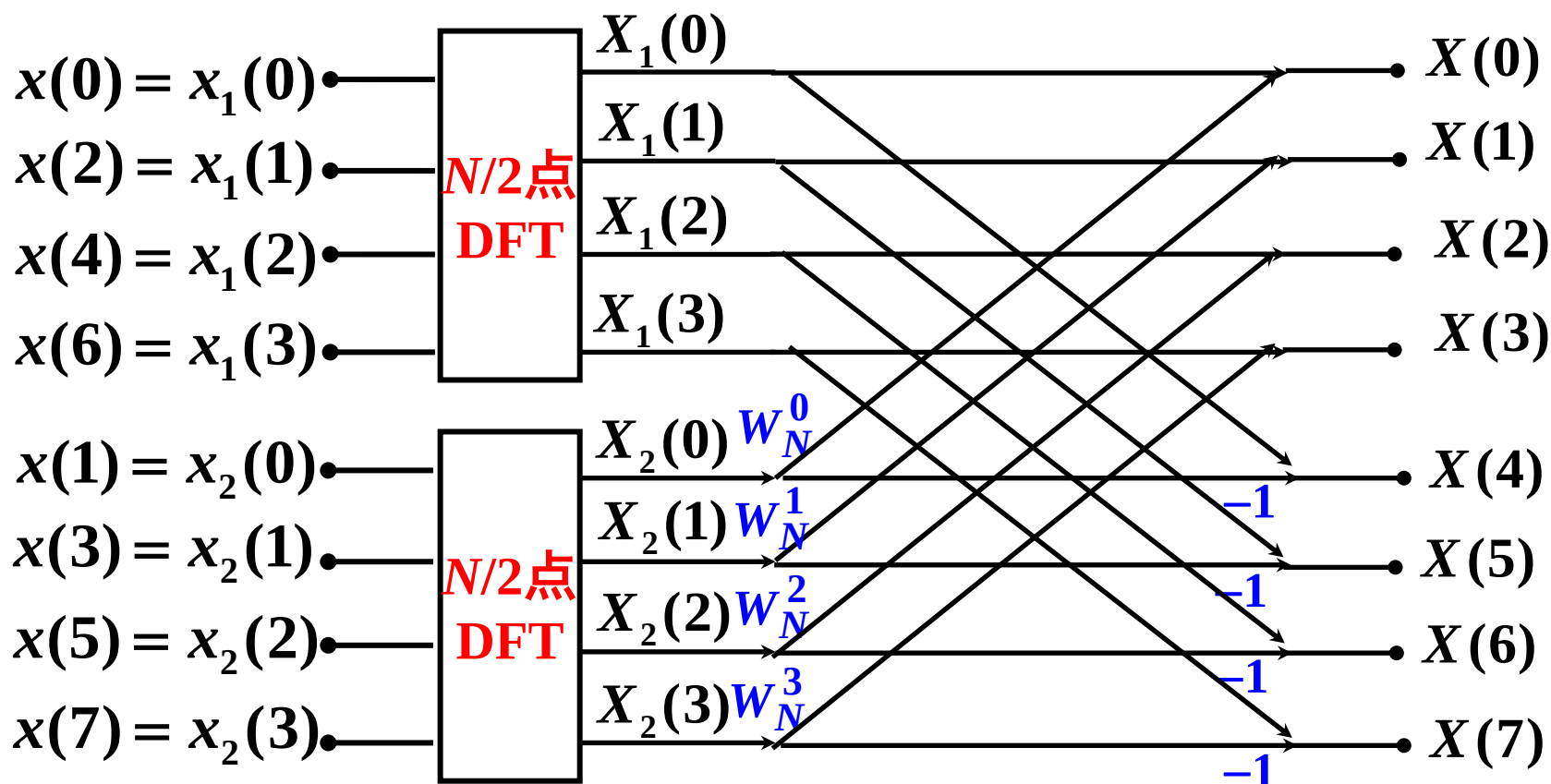
35

➤ 基本思想

以 8 点长序列为例:

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$X(k + \frac{N}{2}) = X_1(k) - W_N^k X_2(k)$$



➤ 运算量分析

分解一次后的运算量：

	复数乘法	复数加法
一个 $N/2$ 点 DFT	$(N/2)^2$	$N/2 (N/2 - 1)$
两个 $N/2$ 点 DFT	$N^2/2$	$N (N/2 - 1)$
一个蝶形	1	2
$N/2$ 个蝶形	$N/2$	N
总计	$N^2/2 + N/2$ $\approx N^2/2$	$N(N/2 - 1) + N$ $\approx N^2/2$

运算量减少了近一半。

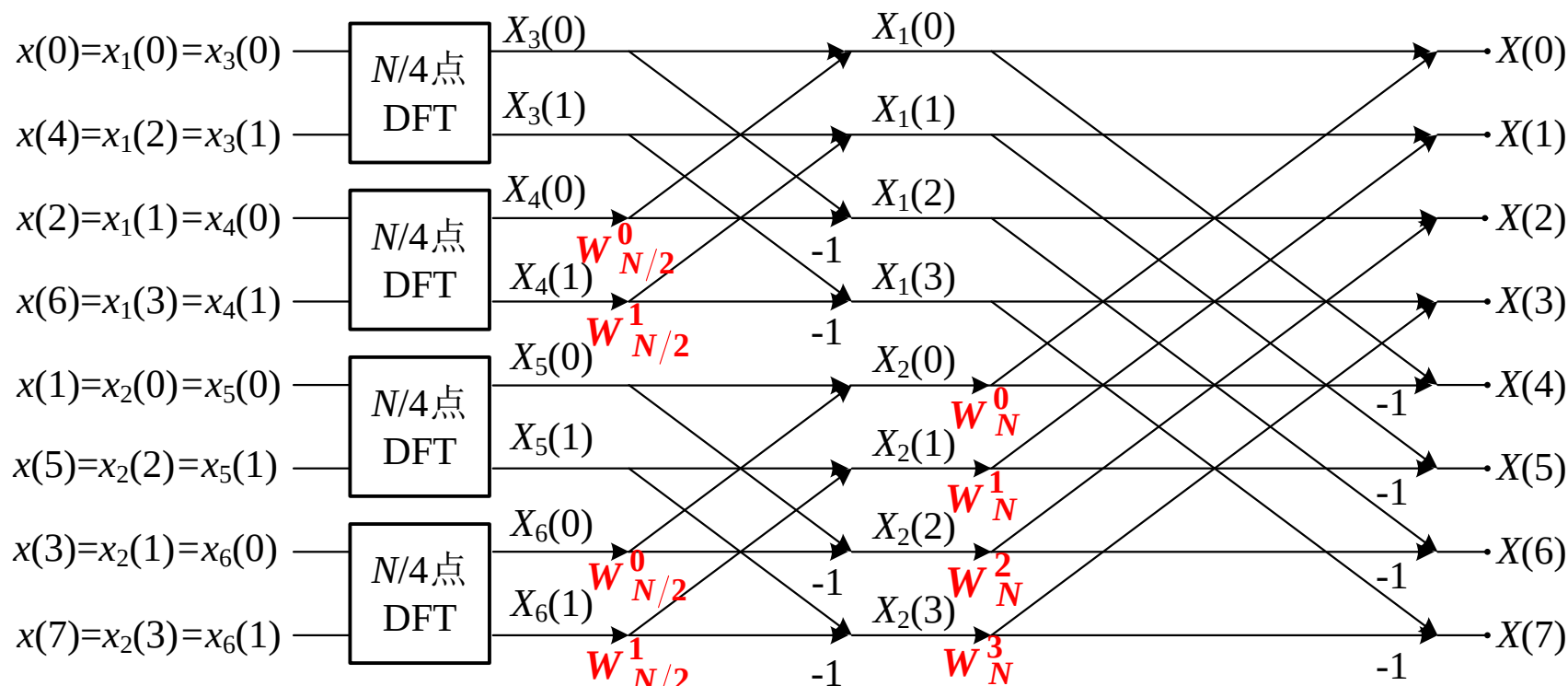
FFT: 按时间抽取



37

➤ 运算量分析 继续分解，直至成为 $N/2$ 个 2 点长序列

$x(0)$	$x(1)$	$x(2)$	$x(3)$	$x(4)$	$x(5)$	$x(6)$	$x(7)$
$x_1(0)$	$x_2(0)$	$x_1(1)$	$x_2(1)$	$x_1(2)$	$x_2(2)$	$x_1(3)$	$x_2(3)$
$x_3(0)$	$x_5(0)$	$x_4(0)$	$x_6(0)$	$x_3(1)$	$x_5(1)$	$x_4(1)$	$x_6(1)$



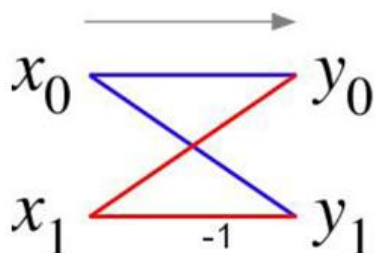
➤ 运算量分析

2点DFT的计算，也用蝶形运算表示：

以 $x_3(n)$ 为例，做 $x_3(n)=[x_3(0), x_3(1)]$ 的 2 点DFT：

$$X_3(\mathbf{k}) = \sum_{l=0}^1 x_3(l) W_2^{lk} = x_3(0) W_2^0 + x_3(1) W_2^k, \quad k = 0, 1$$

$$\begin{cases} X_3(0) = x_3(0) + x_3(1) & = x_3(0) + W_2^0 x_3(1) \\ X_3(1) = x_3(0) + W_2^1 x_3(1) & = x_3(0) - W_2^0 x_3(1) \end{cases}$$



FFT：按时间抽取

39

➤ 运算量分析

序列长度 $N=2^M$ ，共有 M 级蝶形，每级 $N/2$ 个蝶形，共 $MN/2$ 个蝶形，每个蝶形1次复数乘法、2次复数加法。

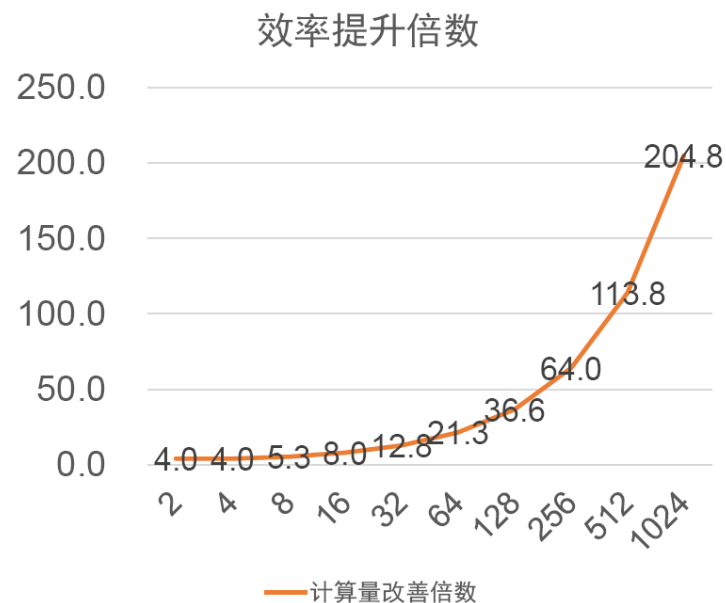
	FFT	DFT
复数乘法	$\frac{NM}{2} = \frac{N}{2} \log_2 N$	N^2
复数加法	$NM = N \log_2 N$	$N(N-1)$

FFT：按时间抽取

40

➤ 运算量分析

序列 长度	DFT	FFT	比较
	乘法 N^2	乘法 $N/2\log_2 N$	
2	4	1	4
8	64	12	5.4
32	1024	80	12.8
64	4096	192	21.3
256	65536	1024	64
1024	1048576	5120	204.8
2048	4194304	11264	372.4
2^{20}	$\approx 10^{12}$	$\approx 10^7$	$\approx 10^5$



➤ 运算量分析

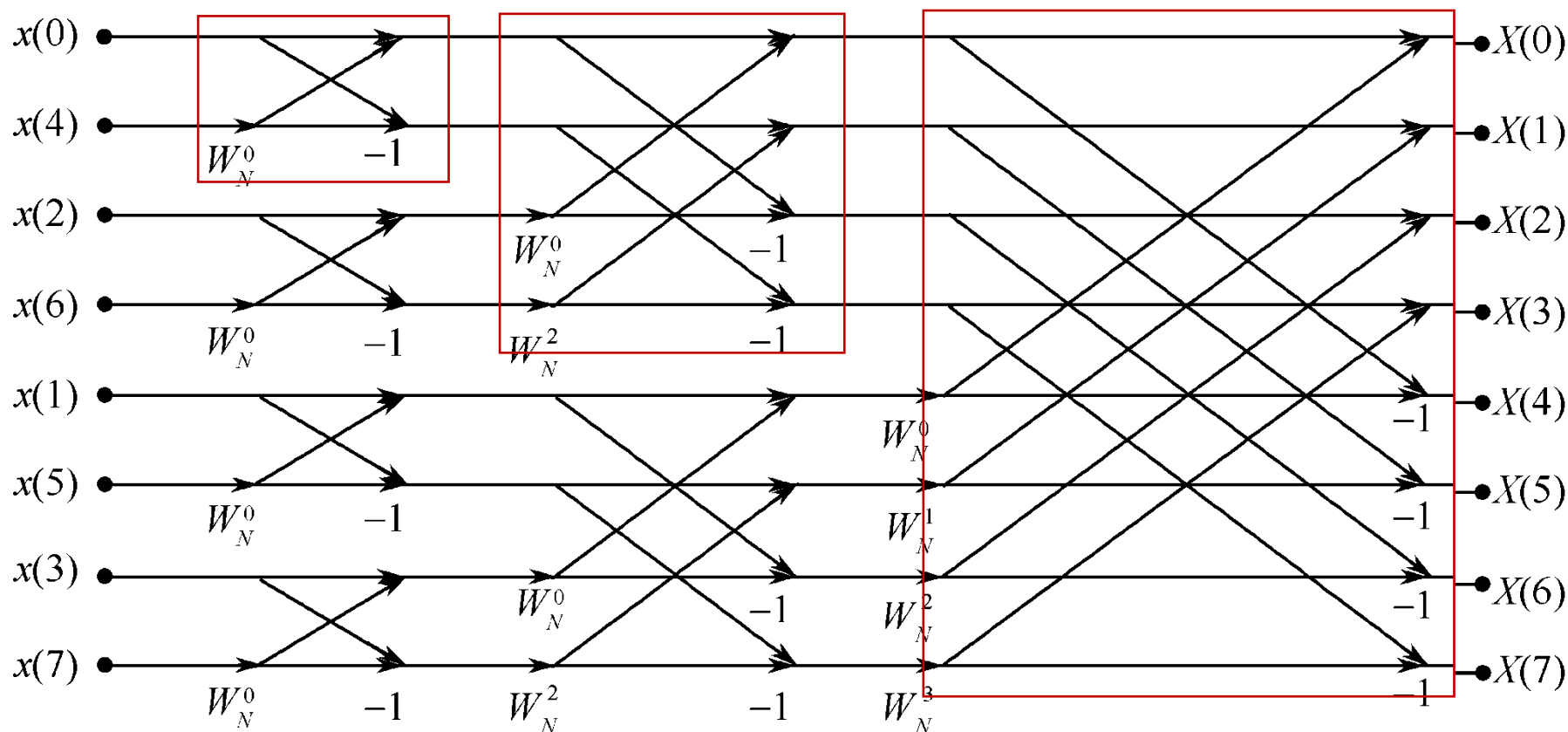
例 用FFT算法处理一副 $N \times N$ 点的二维图像，如用每秒可做10万次复数乘法的计算机，当 $N=1024$ 时，问需要多少时间（不考虑加法运算时间）？

解：当计算点数 $N=1024^2 = 1048576$ 时，FFT算法处理一副二维图像所需复数乘法约为 $\frac{N}{2} \log_2 N \approx 10^7$ 次，为直接计算DFT所需时间（ $N^2 = 1.1 \times 10^{12}$ 次）的10万分之一。

DFT计算需要3054.2小时，FFT运算仅需要105秒（<2分钟）

➤ 算法特点

$x(0)$ $x(1)$ $x(2)$ $x(3)$ $x(4)$ $x(5)$ $x(6)$ $x(7)$
 $x_1(0)$ $x_2(0)$ $x_1(1)$ $x_2(1)$ $x_1(2)$ $x_2(2)$ $x_1(3)$ $x_2(3)$
 $x_3(0)$ $x_5(0)$ $x_4(0)$ $x_6(0)$ $x_3(1)$ $x_5(1)$ $x_4(1)$ $x_6(1)$

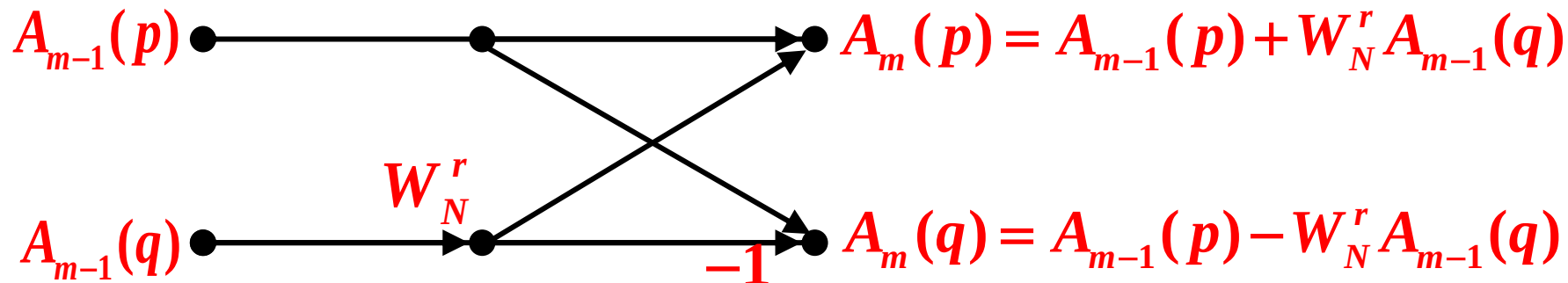
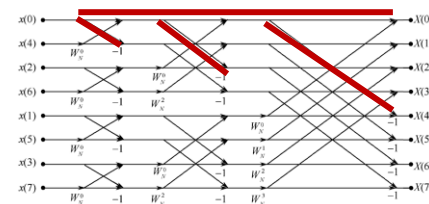


➤ 算法特点

(1) 同址运算（同位运算、原位运算）

将蝶形运算的结果仍然保存在原输入量的存储单元中

(2) 蝶形运算的规律



第 m 级蝶式，节点间距离： $q - p = 2^{m-1}$

➤ 算法特点

旋转因子的规律

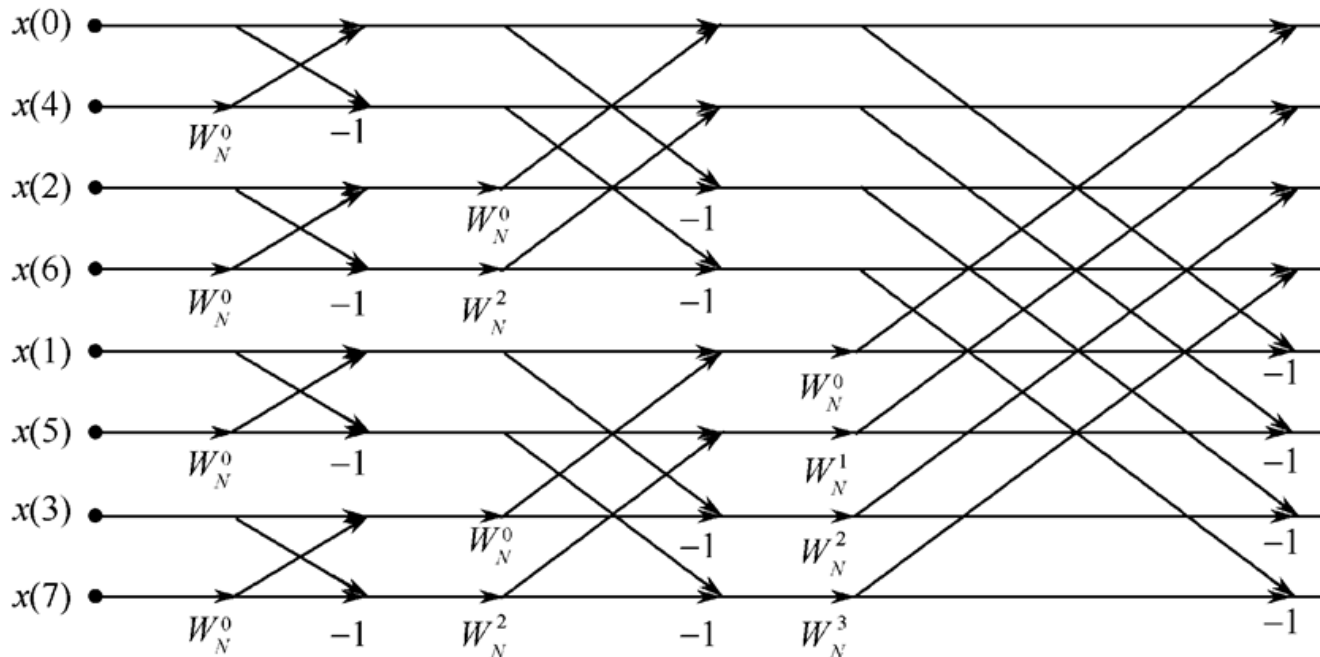
作用	旋转因子	旋转因子的等效表示
2点DFT	$W_2^i, i = 0$	W_N^0
4点→2点	$W_4^i, i = 0, 1$	$W_N^r, r = 0, \frac{N}{4}$
8点→4点	$W_8^i, i = 0, 1, 2, 3$	$W_N^r, r = 0, \frac{N}{8}, \frac{2N}{8}, \frac{3N}{8}$
.....		
$N/2$ 点 → $N/4$ 点	$W_{N/2}^i, i = 0, 1, \dots, \frac{N}{4} - 1$	$W_N^r, r = 0, 2, 4, \dots, \frac{N}{2} - 2$
N 点 → $N/2$ 点	$W_N^i, i = 0, 1, \dots, \frac{N}{2} - 1$	$W_N^r, r = 0, 1, \dots, \frac{N}{2} - 1$

➤ 算法特点

旋转因子的规律

第 m 级蝶形，需要 2^{m-1} 个旋转因子，分别为：

$$W_N^r, r = 2^{M-m} \cdot i, \quad (i = 0, 1, \dots, 2^{m-1} - 1)$$

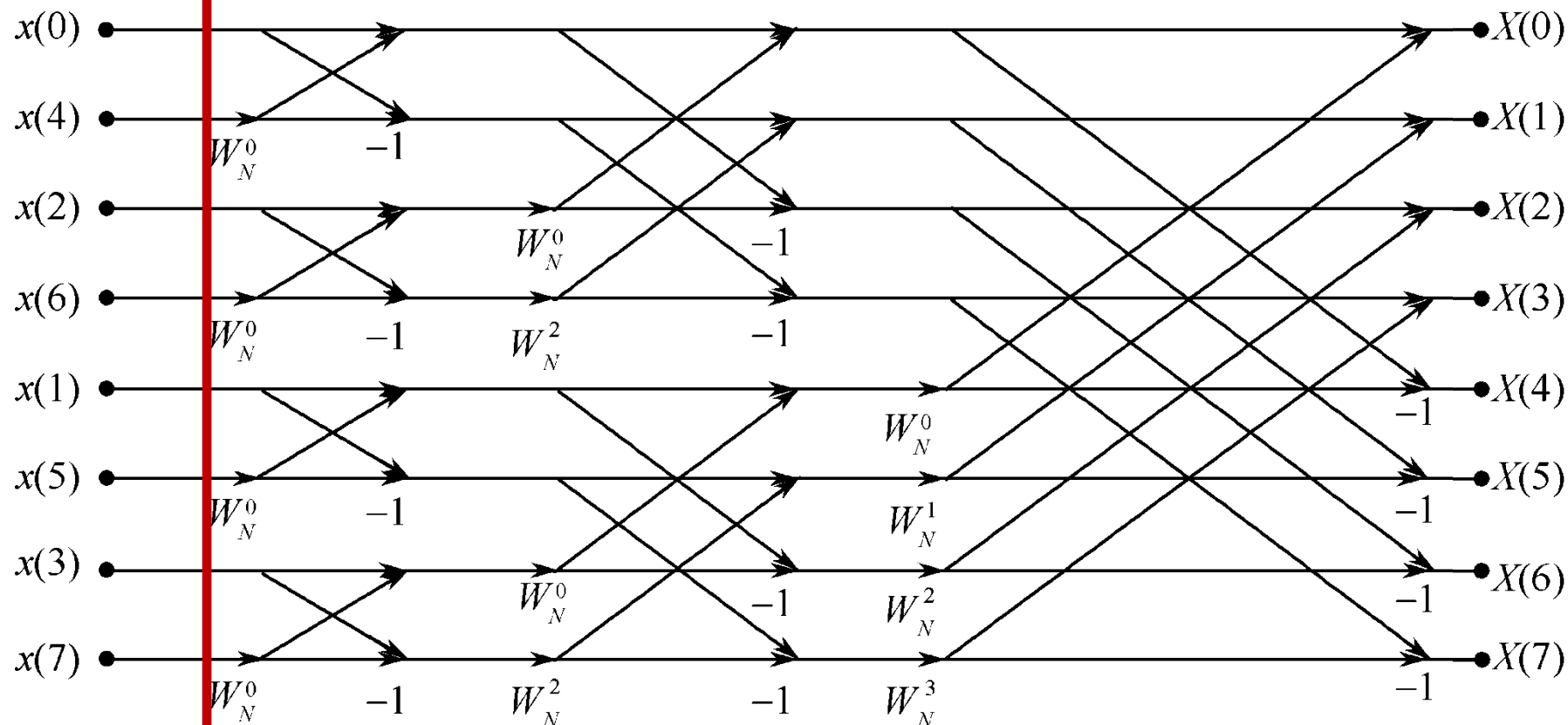


FFT: 按时间抽取

46

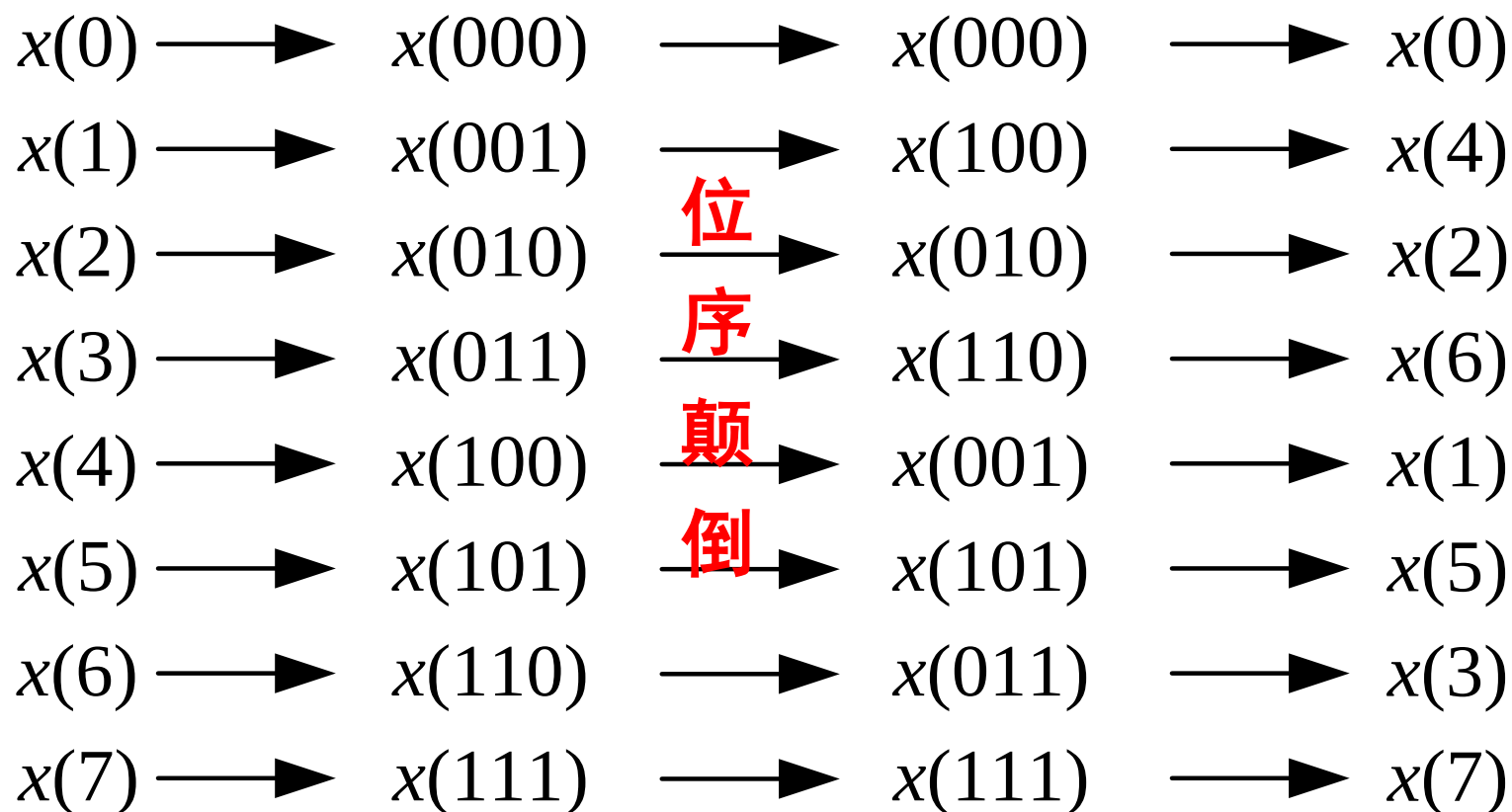
➤ 算法特点

输入倒位序



➤ 算法特点

输入倒位序

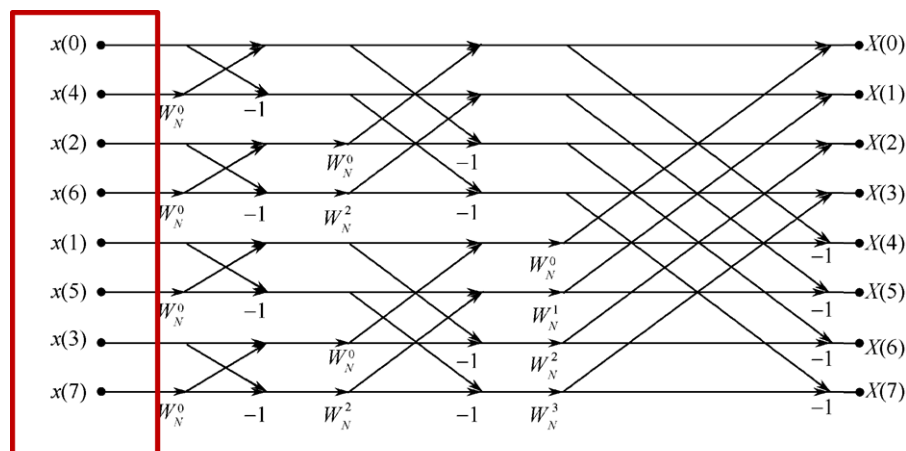


➤ 算法特点

输入倒位序

倒位序，是将二进制数的最高有效位到最低有效位的位序进行**颠倒排列**而得到的二进制数。

DIT-FFT算法中，对时域序列按照序号的**奇偶**进行分解，造成输入序列的序号按照**倒位序**排列。



➤ 小结 DIT (**D**ecimation-**I**n-**T**ime) -FFT

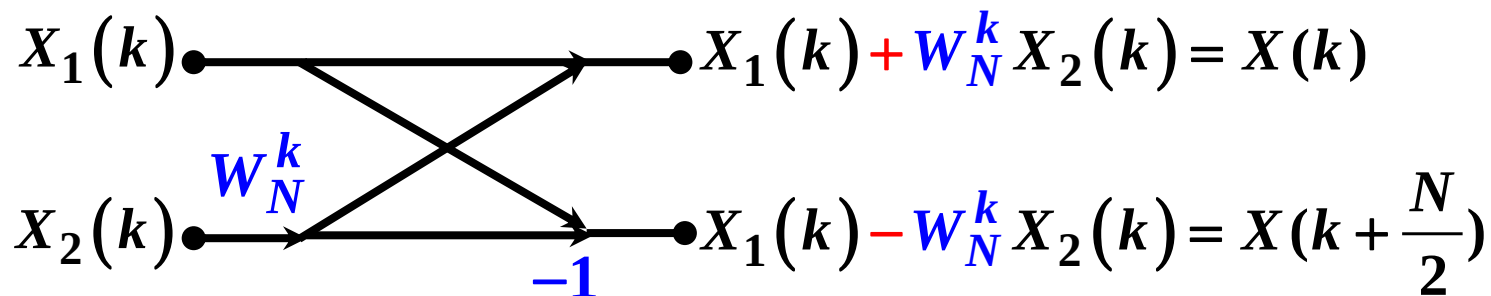
1. 算法原理

2. 运算量分析

3. 算法特点

◆ 怎么分？——将序列 $x(n)$ 按序号 n 的奇偶分解；

◆ 怎么合？——按蝶形运算组合，得到频域 $X(k)$ 的前一半和后一半；



➤ 小结 DIT (**D**ecimation-**I**n-**T**ime) -FFT

1. 算法原理

2. 运算量分析

3. 算法特点

序列长度 $N=2^M$ ，共分解为 $MN/2$ 个蝶形，需要 $MN/2$ 次复数乘， MN 次复数加。

➤ 小结 DIT (**D**ecimation-**I**n-**T**ime) -FFT

1. 算法原理

2. 运算量分析

3. 算法特点

◆ 同址运算（同位运算、原位运算）；

◆ 蝶形运算；

◆ 旋转因子的规律

◆ 输入倒位序。

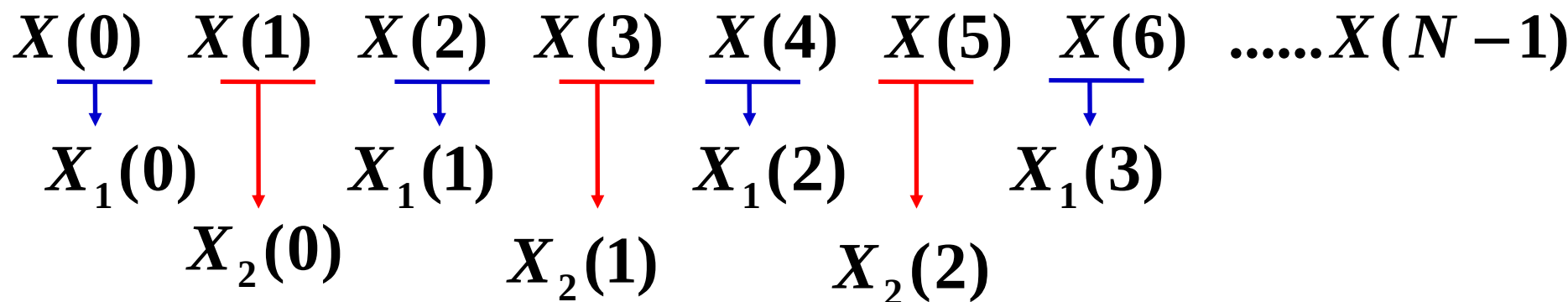
快速傅里叶变换

按频率抽取

前面是倒序输入
是否可以正序输入

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & W_4^1 & -1 & -W_4^1 \\ 1 & -1 & 1 & -1 \\ 1 & -W_4^1 & -1 & W_4^1 \end{bmatrix} \cdot \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \end{bmatrix}$$

将序列 $X(n)$ 按 n 的奇偶分成两组：



➤ 算法原理

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{nk} = \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=\frac{N}{2}}^{N-1} x(n) W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} x(n) W_N^{nk} + \sum_{n=0}^{\frac{N}{2}-1} x\left(n + \frac{N}{2}\right) W_N^{\left(n + \frac{N}{2}\right)k} \\ &= \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) W_N^{Nk/2} \right] \cdot W_N^{nk}, \quad k = 0, 1, \dots, N-1 \end{aligned}$$

由于 $W_N^{N/2} = -1$ 故 $W_N^{Nk/2} = (-1)^k$

$$X(k) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + (-1)^k x\left(n + \frac{N}{2}\right) \right] W_N^{nk}, \quad k = 0, 1, \dots, N-1$$


$$x(n) + x\left(n + \frac{N}{2}\right)$$


$$x(n) - x\left(n + \frac{N}{2}\right)$$

➤ 算法原理

$$X(k) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + (-1)^k x\left(n + \frac{N}{2}\right) \right] W_N^{nk}, \quad k = 0, 1, \dots, N-1$$

$$x(n) + x\left(n + \frac{N}{2}\right)$$

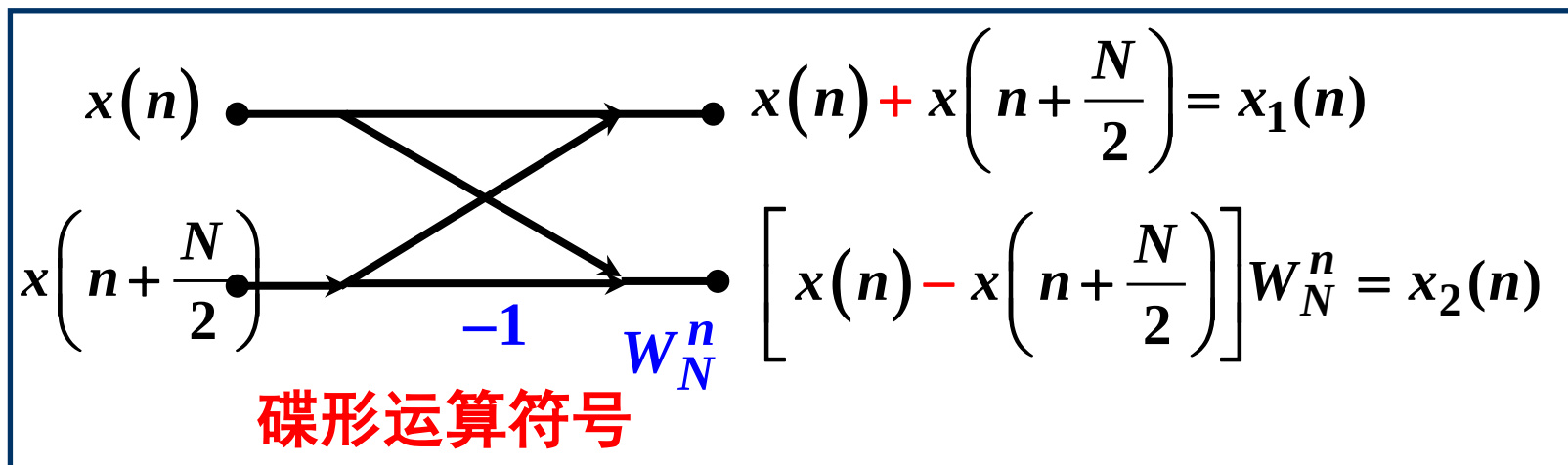
$$x(n) - x\left(n + \frac{N}{2}\right)$$

$$X(2r) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) + x\left(n + \frac{N}{2}\right) \right] W_N^{2nr} = \sum_{n=0}^{\frac{N}{2}-1} \left[\underline{x(n) + x\left(n + \frac{N}{2}\right)} \right] W_{N/2}^{nr}$$

$$X(2r+1) = \sum_{n=0}^{\frac{N}{2}-1} \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^{n(2r+1)} = \left\{ \sum_{n=0}^{\frac{N}{2}-1} \left[\underline{x(n) - x\left(n + \frac{N}{2}\right)} \right] W_N^n \right\} W_{N/2}^{nr}$$

➤ 算法原理

将序列 $x(n)$ 分为前半一半和后半一半，再按蝶形组合成两个 $N/2$ 点长序列；



分别求两个 $N/2$ 点DFT，即得到频域 $X(k)$ 的偶数序号和奇数序号的值。

➤ 算法原理

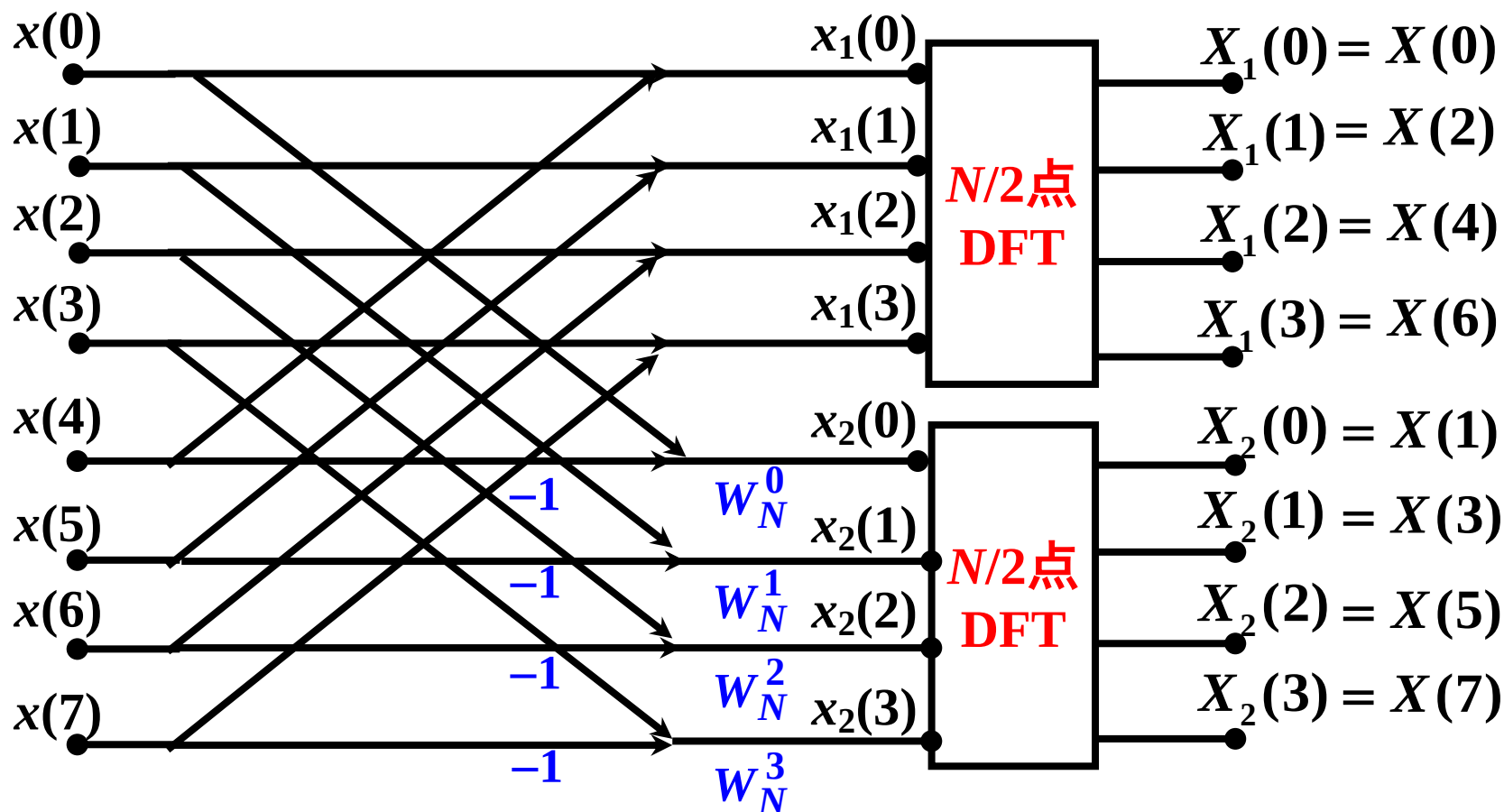
$$\begin{aligned}x_1(n) &= x(n) + x\left(n + \frac{N}{2}\right) \\x_2(n) &= \left[x(n) - x\left(n + \frac{N}{2}\right) \right] W_N^n\end{aligned}\quad n = 0, 1, \dots, \frac{N}{2} - 1$$

$$X(2r) = \sum_{n=0}^{N/2-1} x_1(n) W_{N/2}^{nr}$$

$$r = 0, 1, \dots, \frac{N}{2} - 1$$

$$X(2r+1) = \sum_{n=0}^{N/2-1} x_2(n) W_{N/2}^{nr}$$

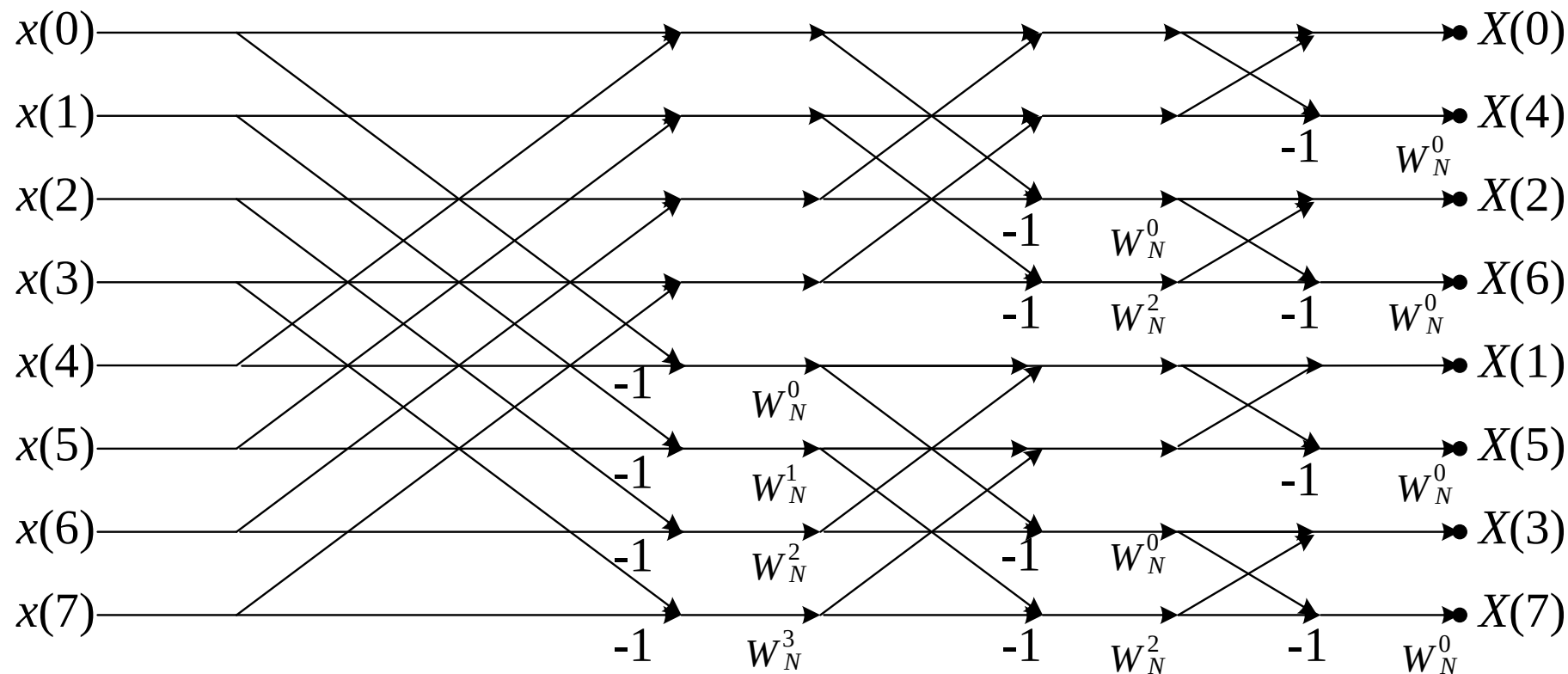
➤ 算法原理



按频率抽取FFT第一次分解 ($N=8$)



➤ 算法原理



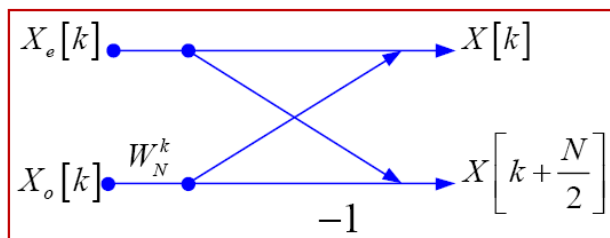
按频率抽取FFT流图 ($N=8$)

FFT：按频率抽取

60

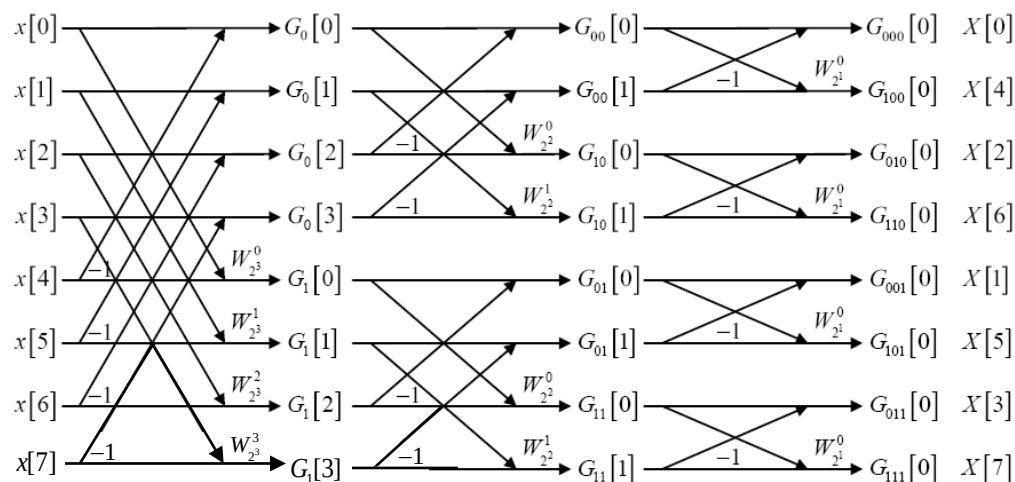
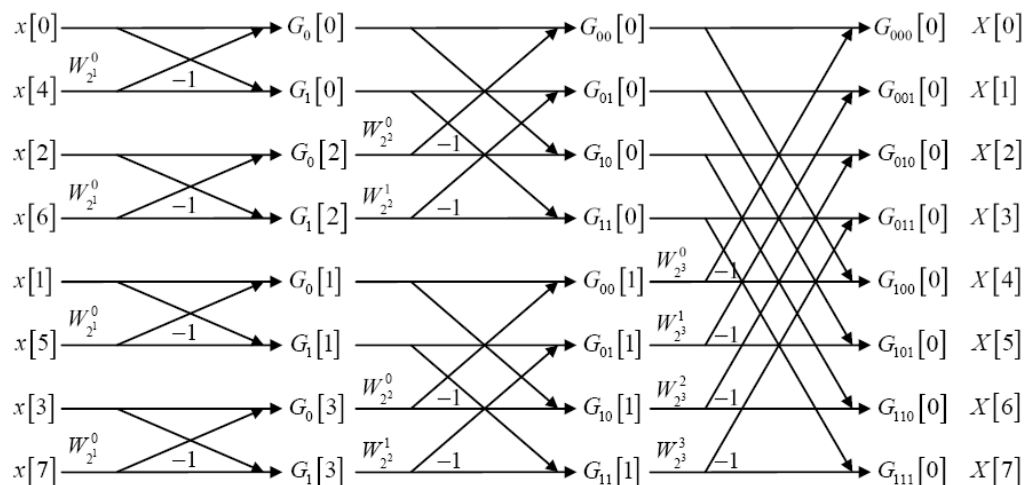
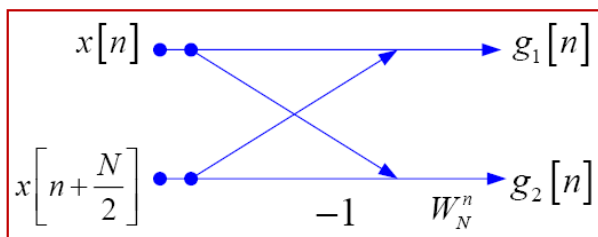
➤ 算法原理

DIT的基本蝶形运算单元



互为转置

DIF的基本蝶形运算单元



➤ 算法分析

- 运算量分析

DIF-FFT 算法的运算量与 DIT-FFT 完全相同。

- 算法特点

- (1) 同址运算（同位运算、原位运算）

- (2) 蝶形运算

- (3) 旋转因子的规律

- (4) 倒位序：输入自然顺序，输出倒位序

FFT：按频率抽取

62

➤ 算法分析

	按时间抽取（DIT）	按频率抽取（DIF）
同址运算	都具备该特点	
运算量	相同 复数乘法 复数加法	$\frac{NL}{2} = \frac{N}{2} \log_2 N$ $NL = N \log_2 N$
输入/输出顺序	输入倒位序，输出自然顺序	输入自然顺序，输出倒位序
蝶形运算	先乘旋转因子，后加减 第m级节点间距离： 2^{m-1}	先加减，后乘旋转因子 第m级节点间距离： 2^{L-m}

二者互为转置

快速傅里叶变换小结

➤ 小结

- 快速傅里叶变换仅仅是计算机运算方式上的优化（结果的精度也有略微的提升，直接算离散傅里叶会积累精度误差）
- FFT的基本思想是把原始的N点序列，依次分解成一系列的短序列。充分利用DFT计算式中指数因子所具有的对称性质和周期性质，进而求出这些短序列相应的DFT并进行适当组合，达到删除重复计算，减少乘法运算和简化结构的目的

➤ 算法实现的一点讨论

- 1、算法复杂度
 - 之前的分析我们只强调了乘法和加法次数，具体实现还有其他问题需要考虑，包括处理器架构、可用的指令集、存储旋转因子的数据结构等
- 2、结构规整度
 - 绝大多数DSP处理器通常为单步乘加操作指令、逆位序索引、高度并行性等专门进行了优化，使得算法的**结构规整性**与计算量具有同样重要的地位，基2、基4同址计算的结构更具有优势
 - 结构规整性对在矢量处理器、大规模集成电路上实现FFT十分重要

➤改进的FFT

1. 混合基算法 (N 为复合数时的FFT)

若 N 是一个复合数，可分解为 $N=r_1r_2\dots r_L$

若 $r_1 = r_2 = \dots = r_L = r$ ，称为 **基 r 算法**；

若 $r=2$ ，称为**基 2 算法**；

若 $r=4$ ，称为**基 4 算法**；

若各个 r_i 不相同，称为**混合基（分裂基）算法**。

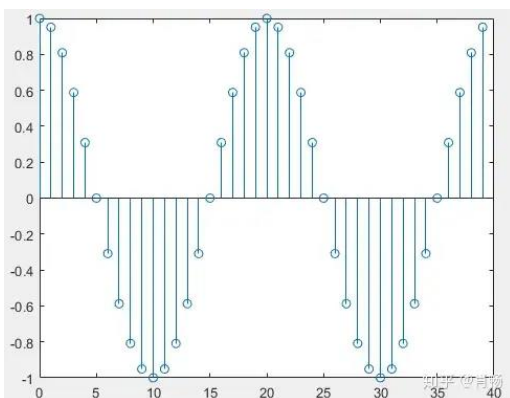
➤改进的FFT

2. Goertzel算法

在某些应用场合，只需要计算很少几个频率点的频谱值。此时Goertzel算法（卡泽尔算法）的效率将优于一般的FFT算法。

➤ 从DFT角度理解FFT

在长度为N的离散信号中，针对 $k=(0,1,2,...)$ ，分别找出在长度N内振动k个周期的三角波分量的权值

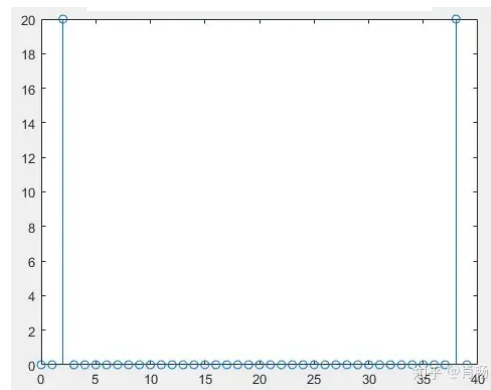


$$\sum_k x[k]y[k]$$

$$X_2 = X_{38} :$$

$$= \sum_{n=0}^{39} \cos(2\pi \frac{2n}{40}) \cos(2\pi \frac{2n}{40}) = 20$$

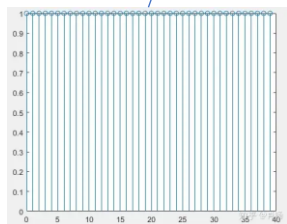
$$X_k = \sum_{n=0}^{N-1} x[n] \cos(2\pi \frac{kn}{N})$$



$$X_k = \sum_{n=0}^{N-1} x[n] e^{-2\pi j \frac{kn}{N}}$$

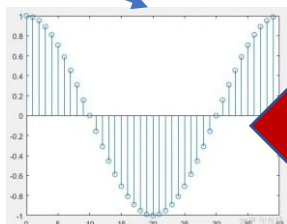
$$X_k = \sum_{n=0}^{N-1} x[n] \cos(2\pi \frac{kn}{N}) - j \sum_{n=0}^{N-1} x[n] \sin(2\pi \frac{kn}{N})$$

对于一个信号，如果只跟余弦函数比较，会损失一些信息，比如相位。需要增加正弦函数分量。



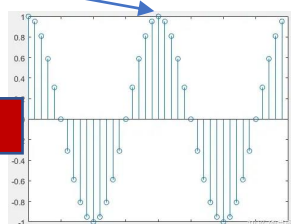
40次采样内振动0个周期

$$\cos(2\pi \frac{0n}{40})$$



40次采样内振动1个周期

$$\cos(2\pi \frac{1n}{40})$$



40次采样内振动2个周期

$$\cos(2\pi \frac{2n}{40})$$

$$\cos(2\pi \frac{3n}{40})$$

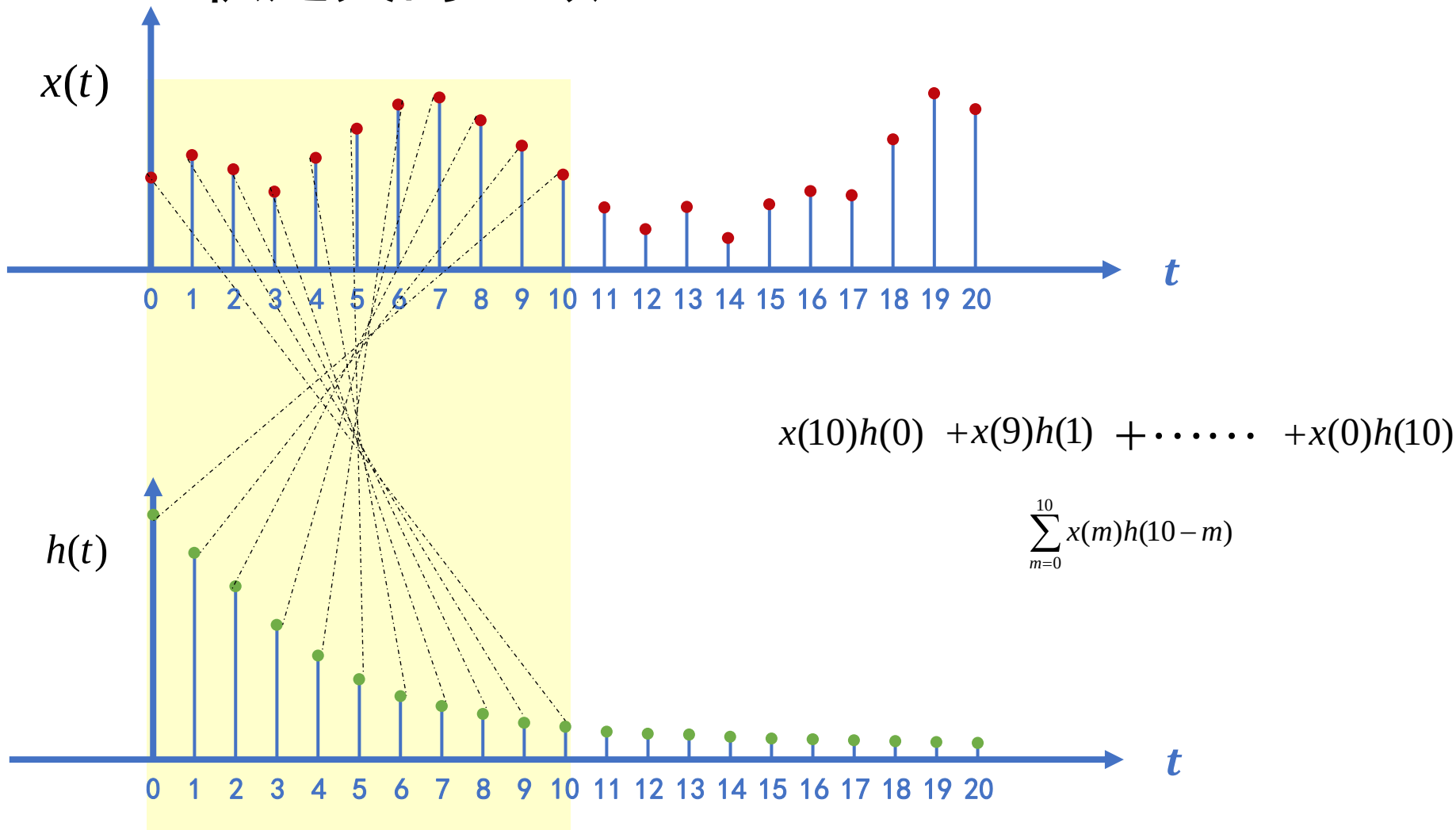
$$\cos(2\pi \frac{4n}{40})$$

.....

信号处理举例

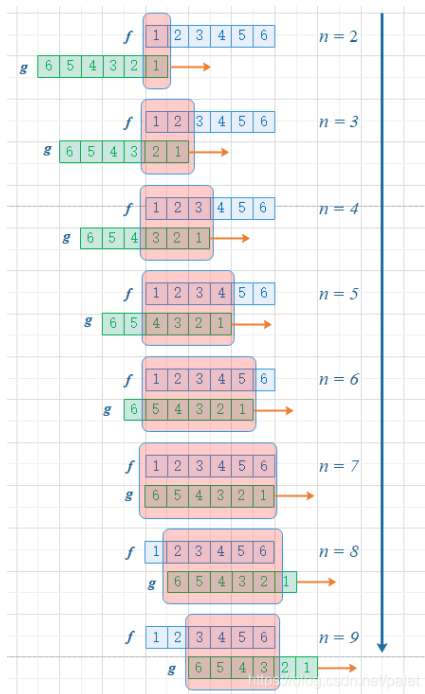
卷积的快速运算

➤ 卷积定义的回顾



➤ 卷积定义的回顾

$$f(n) * g(n) = \sum_{m=-\infty}^{+\infty} f(m)g(n-m)$$



$$\sum_{m=-\infty}^{+\infty} f(m)g(n-m)$$

$$N \geq N_1 + N_2 - 1$$

➤ 卷积定义的回顾

$x_1(n)$ 长度为 N_1 , $x_2(n)$ 长度为 N_2

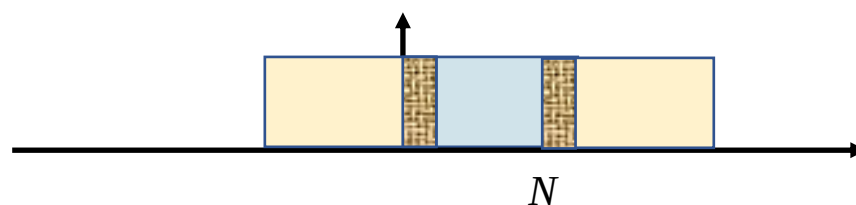
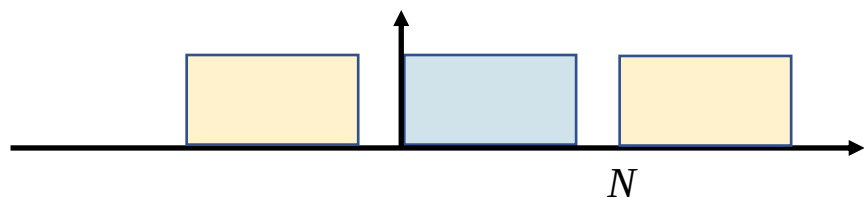
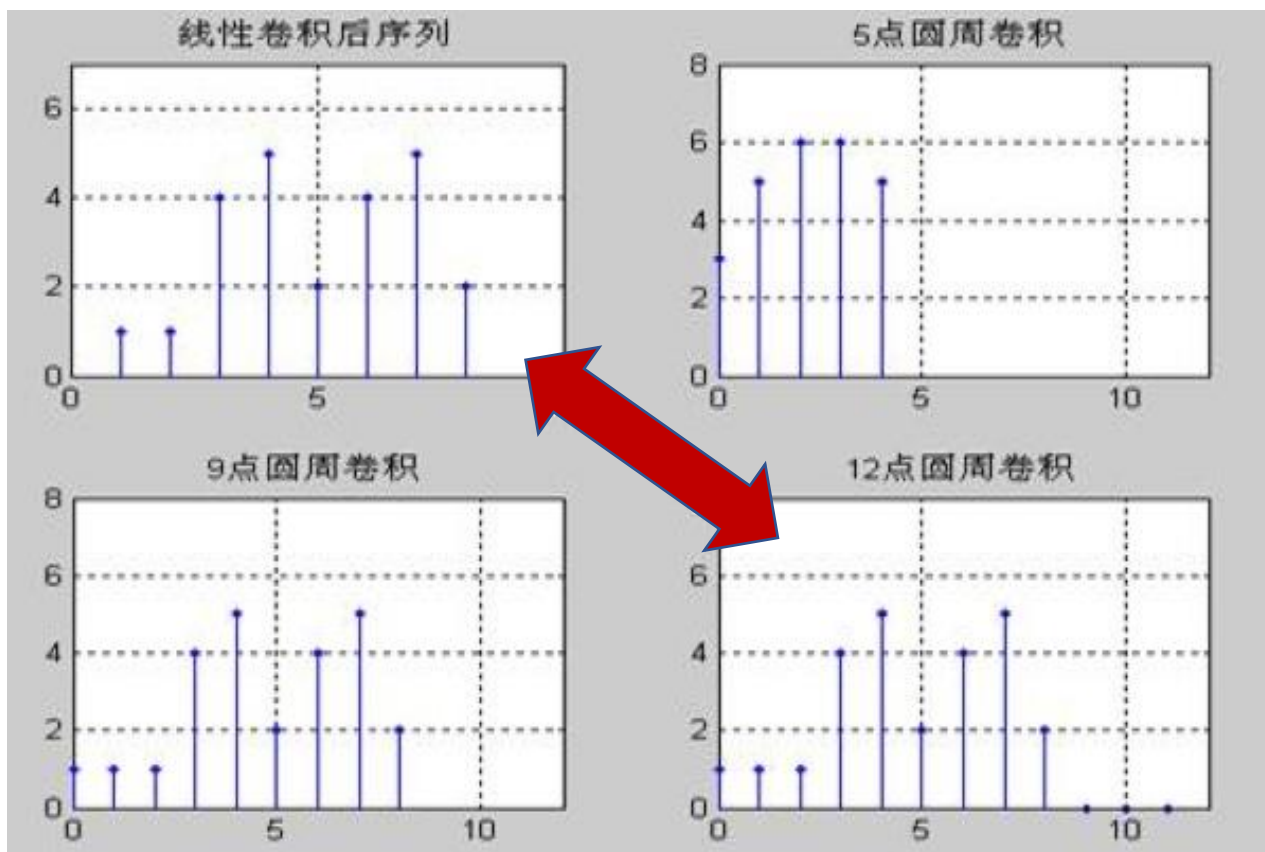
线性卷积: $y_l(n) = x_1(n) * x_2(n)$

N 点圆周卷积: $x_1(n) \textcircled{N} x_2(n) = \sum_{r=-\infty}^{\infty} y_l(n+rN)R_N(n)$

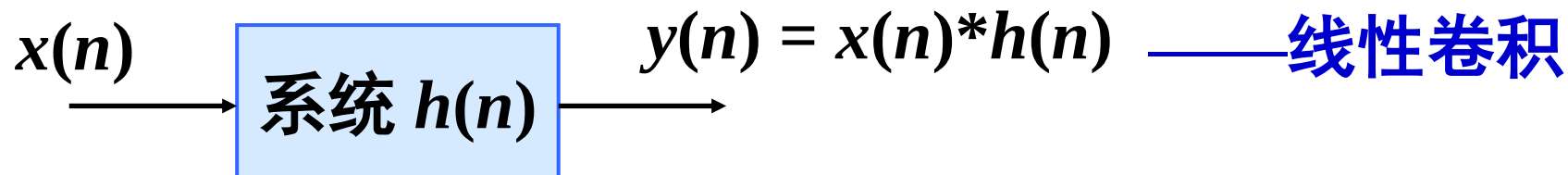
圆周卷积是线性卷积周期延拓后的主值序列

当 $N \geq N_1 + N_2 - 1$ 时, 圆周卷积主值和线性卷积的结果相同。

➤ 卷积定义的回顾



➤问题背景



$x(n)$ 长度为 L , $h(n)$ 长度为 M

直接计算线性卷积, 需要 $L \times M$ 次乘法

DFT的卷积定理

$$x(n) \textcircled{N} h(n) \xleftrightarrow[N]{DFT} X(k)H(k)$$

➤回顾：圆周卷积与线性卷积的关系

$x_1(n)$ 长度为 L ， $x_2(n)$ 长度为 M

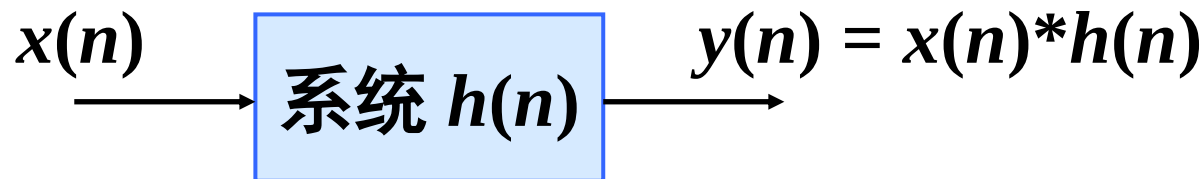
线性卷积： $y_l(n) = x_1(n) * x_2(n)$

N 点圆周卷积： $x_1(n) \circledR x_2(n) = \sum_{r=-\infty}^{\infty} y_l(n + rN) R_N(n)$

圆周卷积是线性卷积周期延拓后的主值序列。

当 $N \geq L + M - 1$ 时， 圆周卷积和线性卷积的结果相同。

➤快速卷积算法



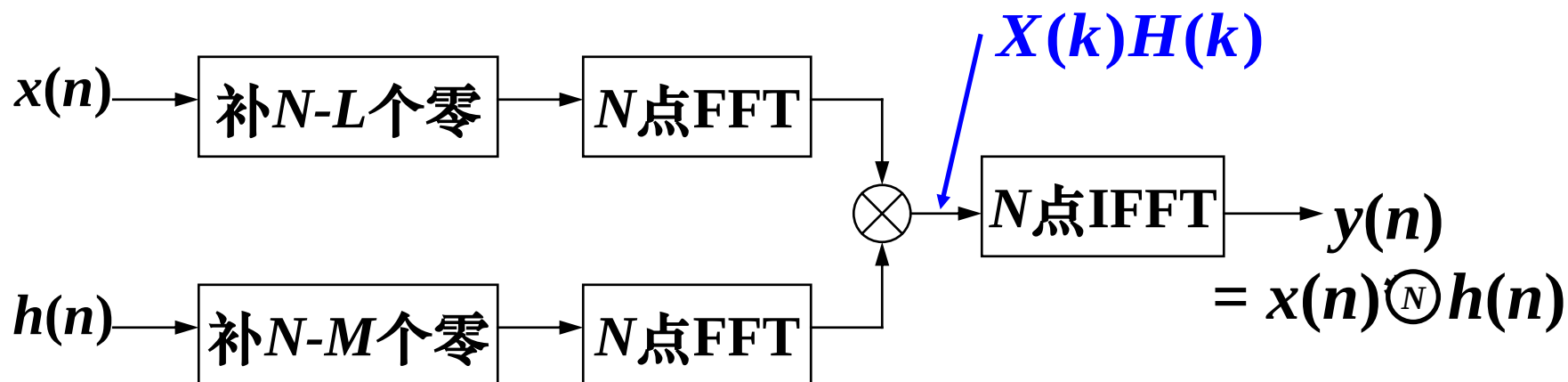
$x(n)$ 长度为 L ， $h(n)$ 长度为 M

直接计算线性卷积，需要 $L \times M$ 次乘法

借助FFT算法，实现线性卷积的运算，从而节省运算量，称为快速卷积算法。

➤快速卷积算法

$x(n)$ 长度为 L , $h(n)$ 长度为 M



$$IDFT[X(k)H(k)] = x(n) \circledN h(n)$$

当 $N \geq L+M-1$ 时, $y(n)$ 可以由 $x(n) \circledN h(n)$ 得到。

运算量分析

- 两次 N 点FFT, 一次 N 点IFFT, 共 $3 \times [N \times \log_2(N)]/2 = 1.5N \log_2(N)$;
- 两个FFT结果相乘, 需要 N 次乘法。

►快速卷积算法运算量分析

$x(n)$ 长度为 L ， $h(n)$ 长度为 M

直接计算	快速卷积
ML	$(M + L - 1)[1 + 1.5\log_2(M + L - 1)]$
二者之比： $K_m = \frac{ML}{(M + L - 1)[1 + 1.5\log_2(M + L - 1)]}$	

➤快速卷积算法运算量分析

$x(n)$ 长度为 L ， $h(n)$ 长度为 M

$$K_m = \frac{ML}{(M + L - 1)[1 + 1.5\log_2(M + L - 1)]}$$

(1) 当 $L \approx M$ 时:
$$K_m \approx \frac{M}{5 + 3\log_2 M}$$

M=L	8	32	64	128	256	512	1024	2048	4096
Km	0.286	0.80	1.39	2.46	4.41	8	14.62	26.95	49.95

点数较多时优势明显

➤快速卷积算法运算量分析

$x(n)$ 长度为 L ， $h(n)$ 长度为 M

$$K_m = \frac{ML}{(M + L - 1)[1 + 1.5\log_2(M + L - 1)]}$$

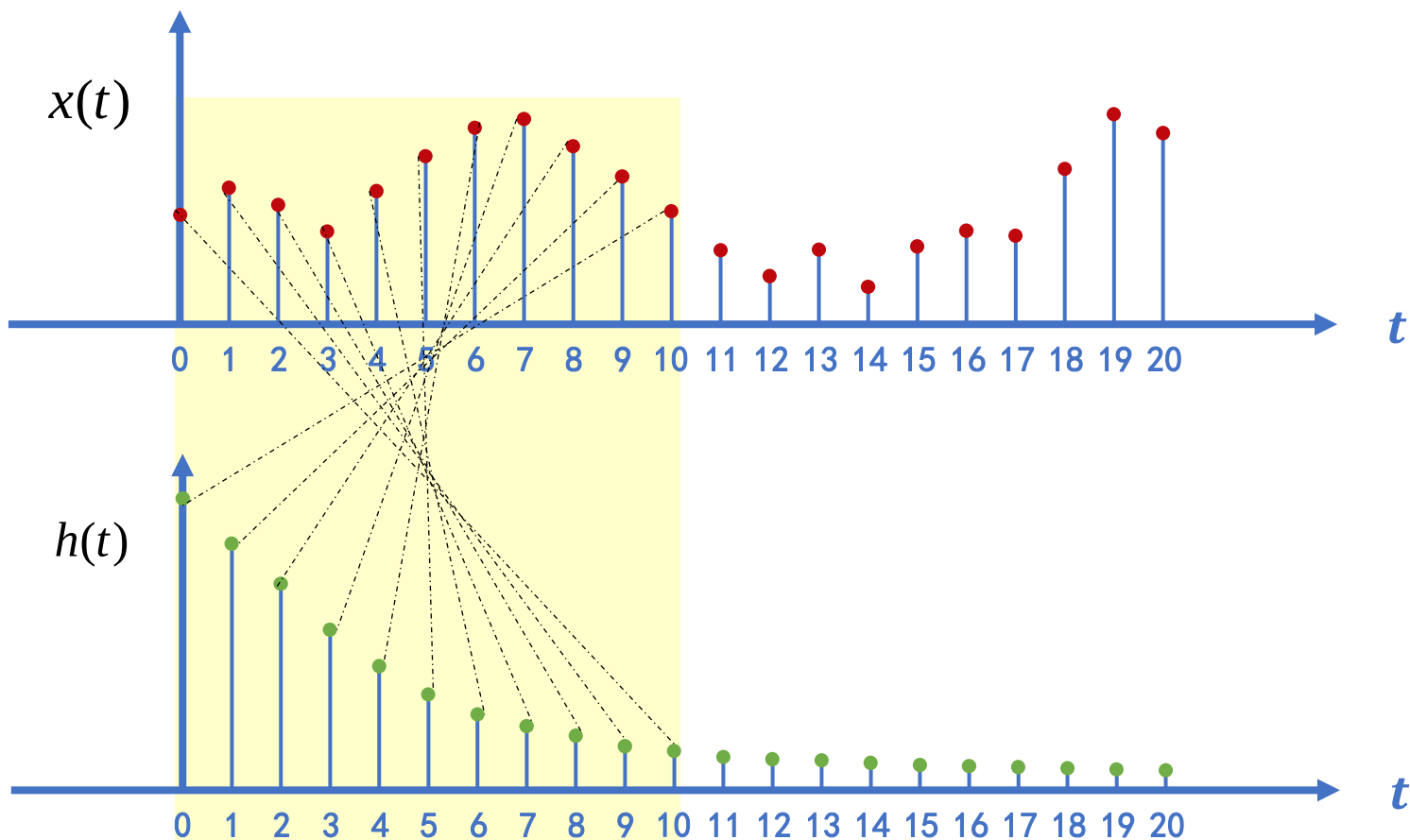
(2) 当 $L \gg M$ 时:
$$K_m \approx \frac{M}{1 + 1.5\log_2 L}$$

$M=64$, $L=1024$, $K_m=4$

L 大时，优势不明显，甚至适得其反

➤快速卷积算法

- 实际应用中， L 可能很大， M 可能比较短
- 太大的 L 影响计算效率

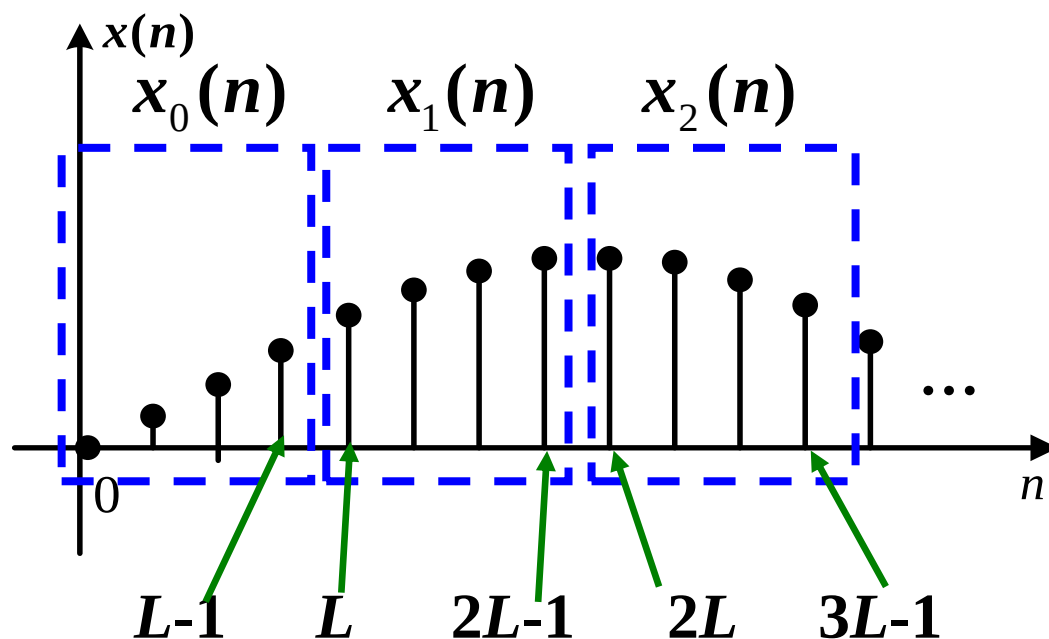


➤改进措施

(1) 重叠相加法

$x(n)$ 很长，分为 K 段，每段长 L ， $h(n)$ 长度为 M

$$x(n) = \sum_{i=0}^{K-1} x_i(n)$$



$$x_i(n) = \begin{cases} x(n), & iL \leq n \leq (i+1)L-1 \\ 0, & \text{others} \end{cases}, i = 0, 1, \dots$$

➤改进措施

(1) 重叠相加法

$x(n)$ 很长，分为 K 段，每段长 L ， $h(n)$ 长度为 M

$$x(n) = \sum_{i=0}^{K-1} x_i(n)$$

$$y(n) = h(n) * x(n) = h(n) * \sum_{i=0}^{K-1} x_i(n) = \sum_{i=0}^{K-1} [h(n) * x_i(n)]$$

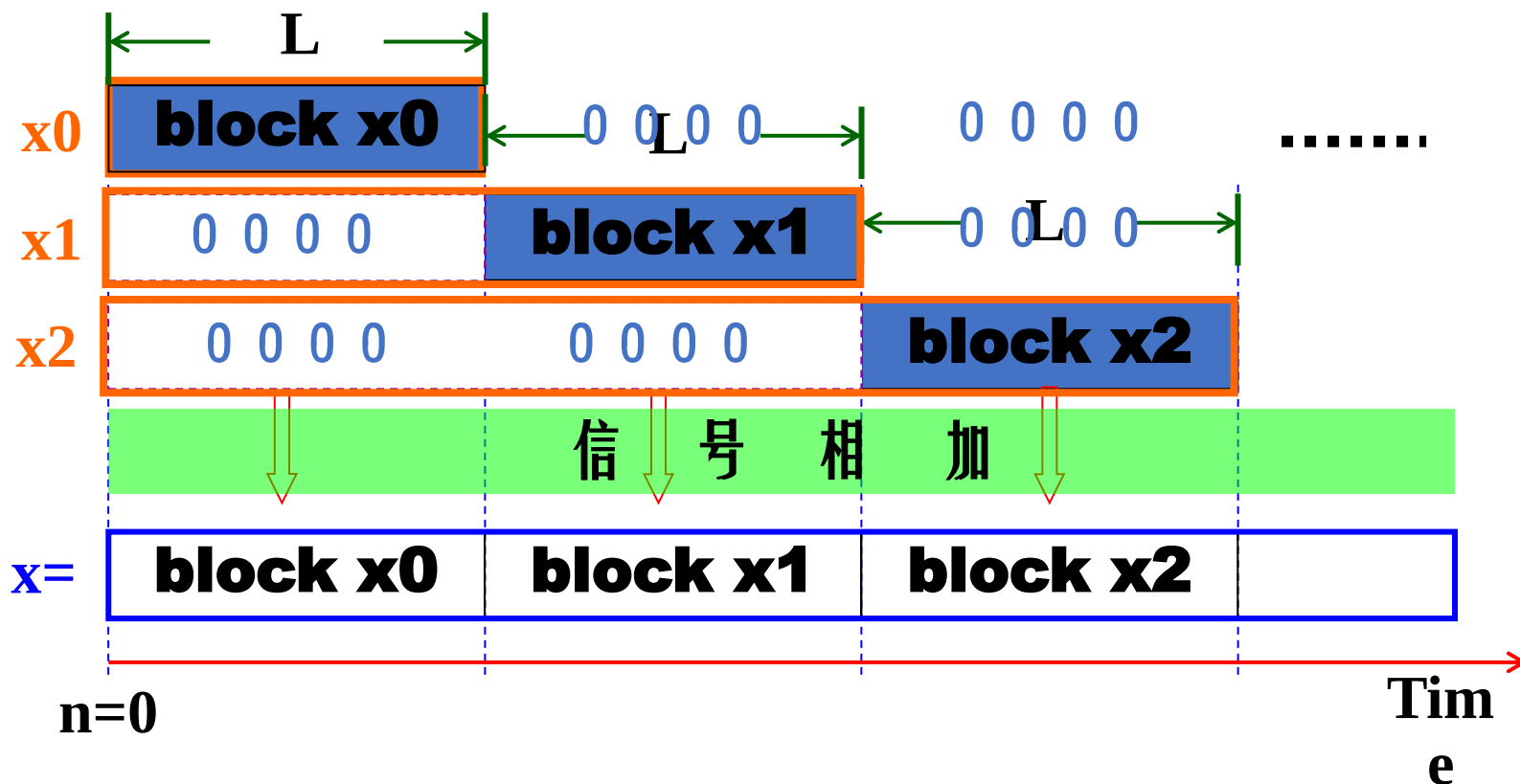
$$N \geq L + M - 1 \quad y_i(n) = h(n) \textcircled{N} x_i(n) = h(n) * x_i(n)$$

➤改进措施

(1) 重叠相加法

信号（序列）的加法运算规则

$$x(n) = \sum_{i=0}^{K-1} x_i(n)$$



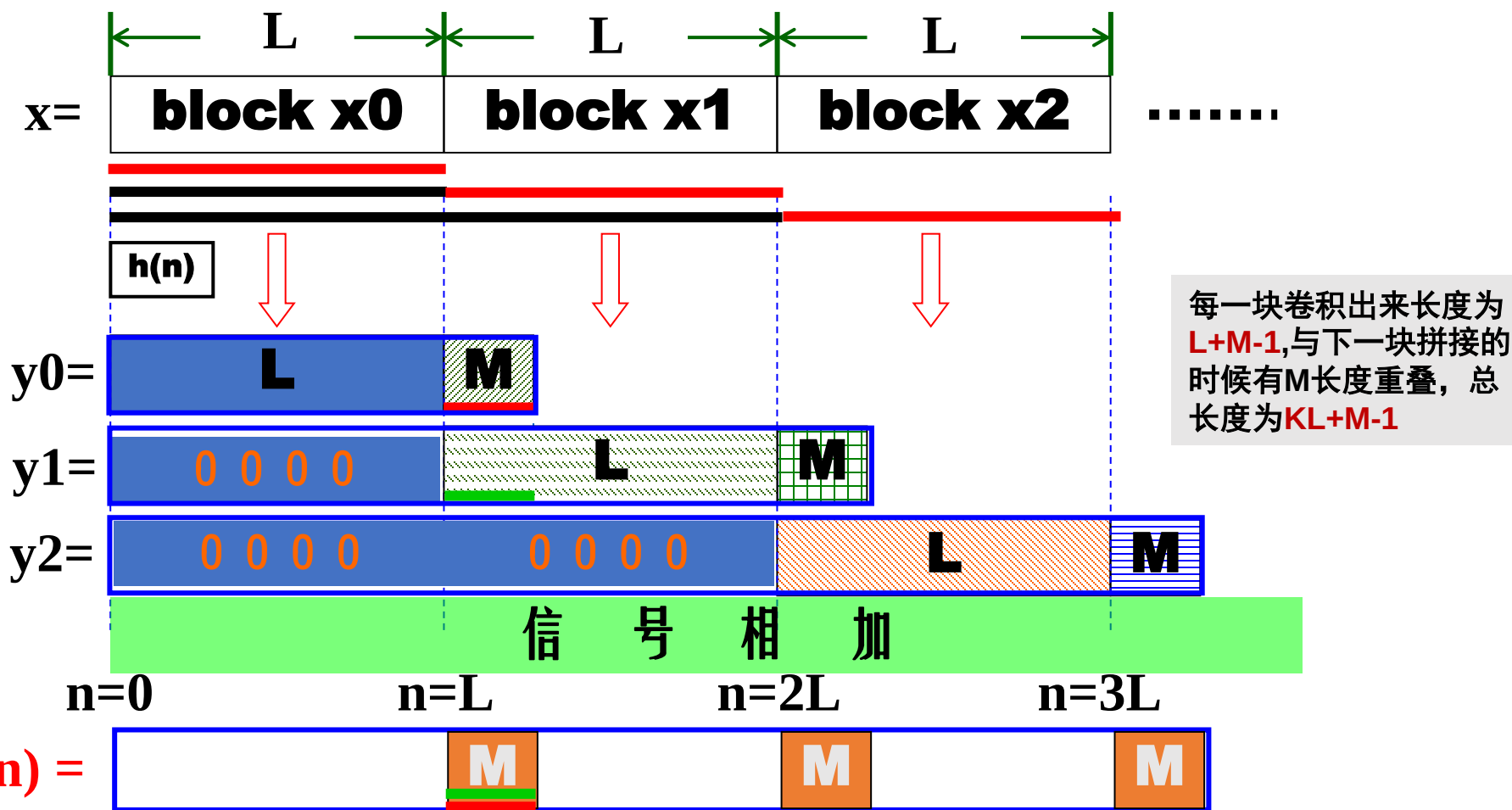
卷积的快速运算

85

改进措施

分段卷积

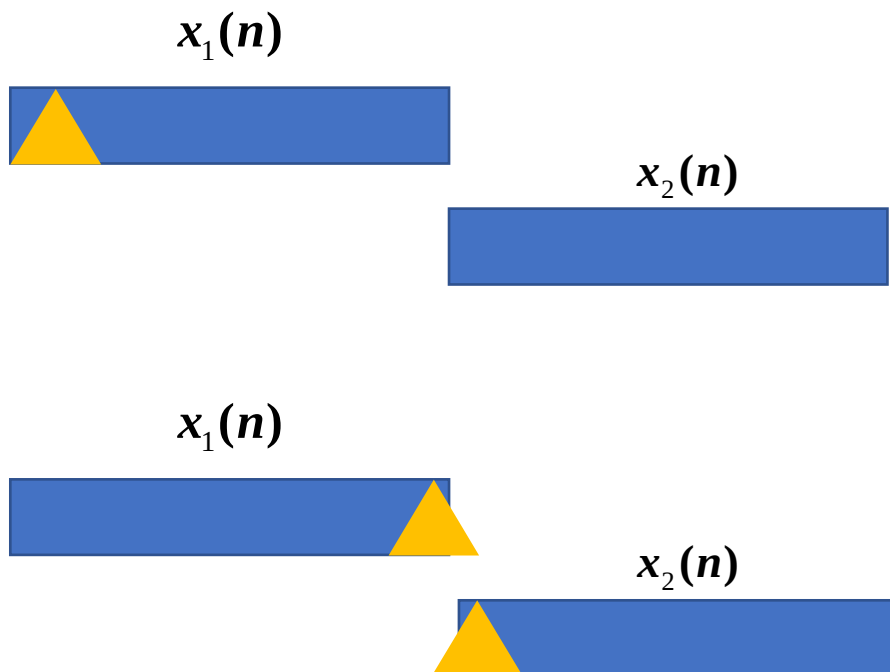
$$y(n) = \underbrace{h(n)}_M * \underbrace{x(n)}_{KL} = \sum_{i=0}^{K-1} \left[\underbrace{h(n) * x_i(n)} \right]$$



➤ 改进措施

$$y(n) = h(n) * x(n) = \sum_{i=0}^{K-1} [h(n) * x_i(n)]$$

(1) 重叠相加法

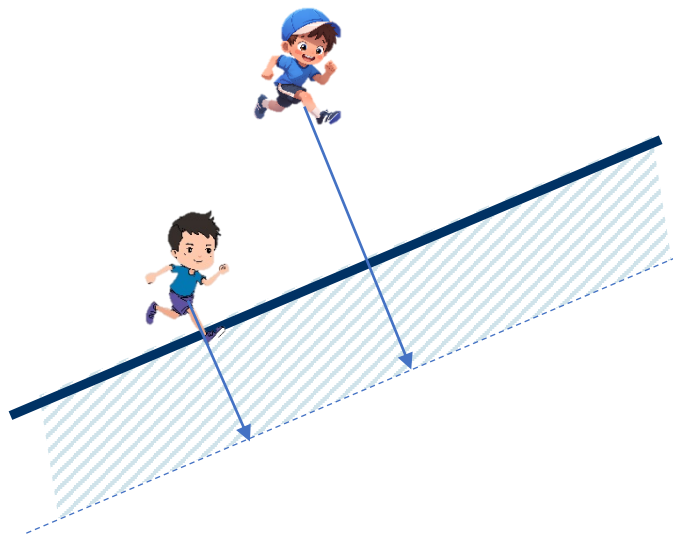


卷积的快速运算

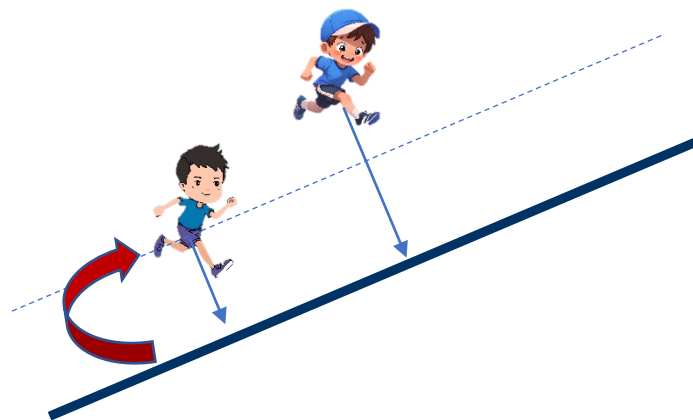
87

➤改进措施

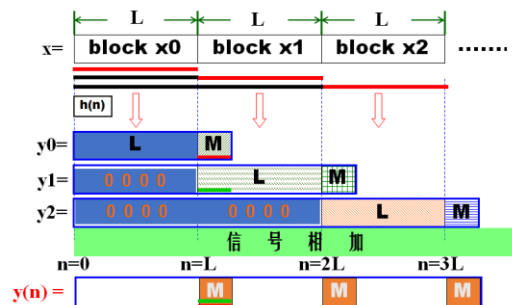
$$y(n) = h(n) * x(n) = \sum_{i=0}^{K-1} [h(n) * x_i(n)]$$



重叠相加法



直接拼接？



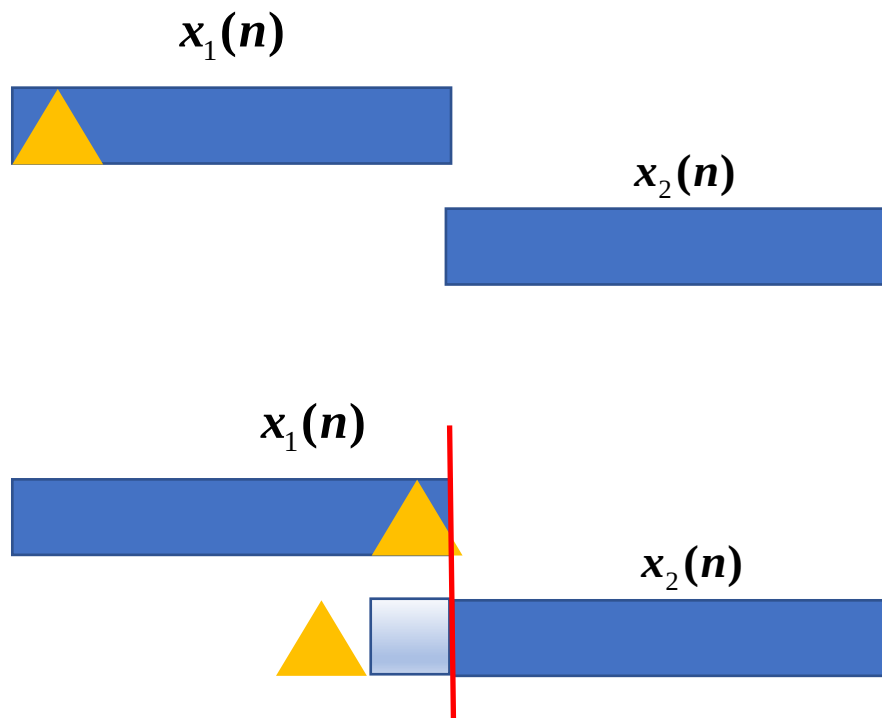
提前预热

卷积的快速运算

88

➤ 改进措施

$$y(n) = h(n) * x(n) = \sum_{i=0}^{K-1} [h(n) * x_i(n)]$$

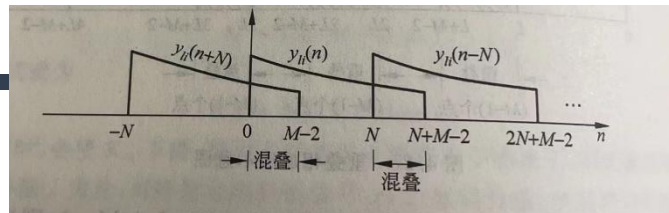


提前预热



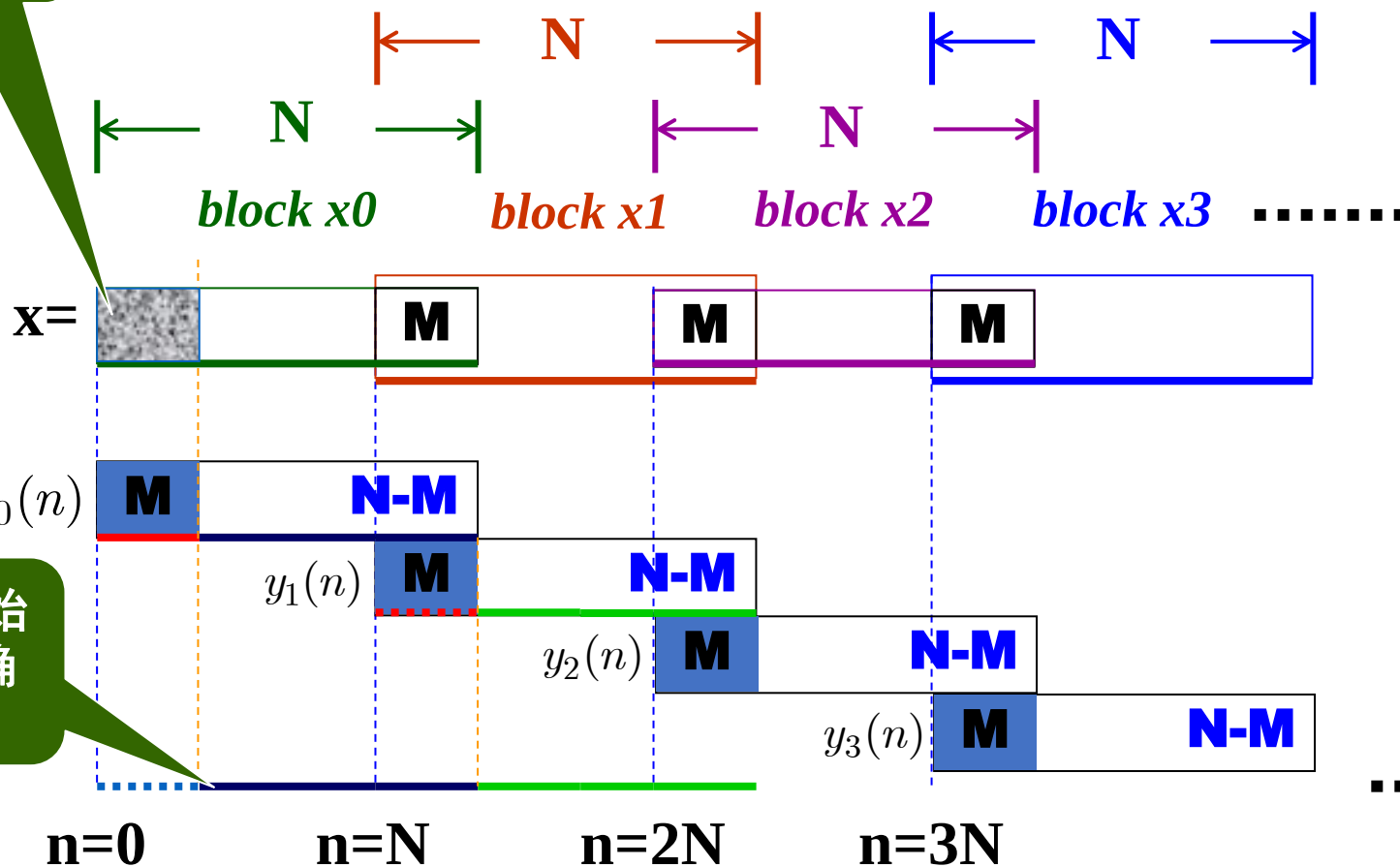
卷积的快速运算

改进措施



序列前面
插入M个0

重叠保留法



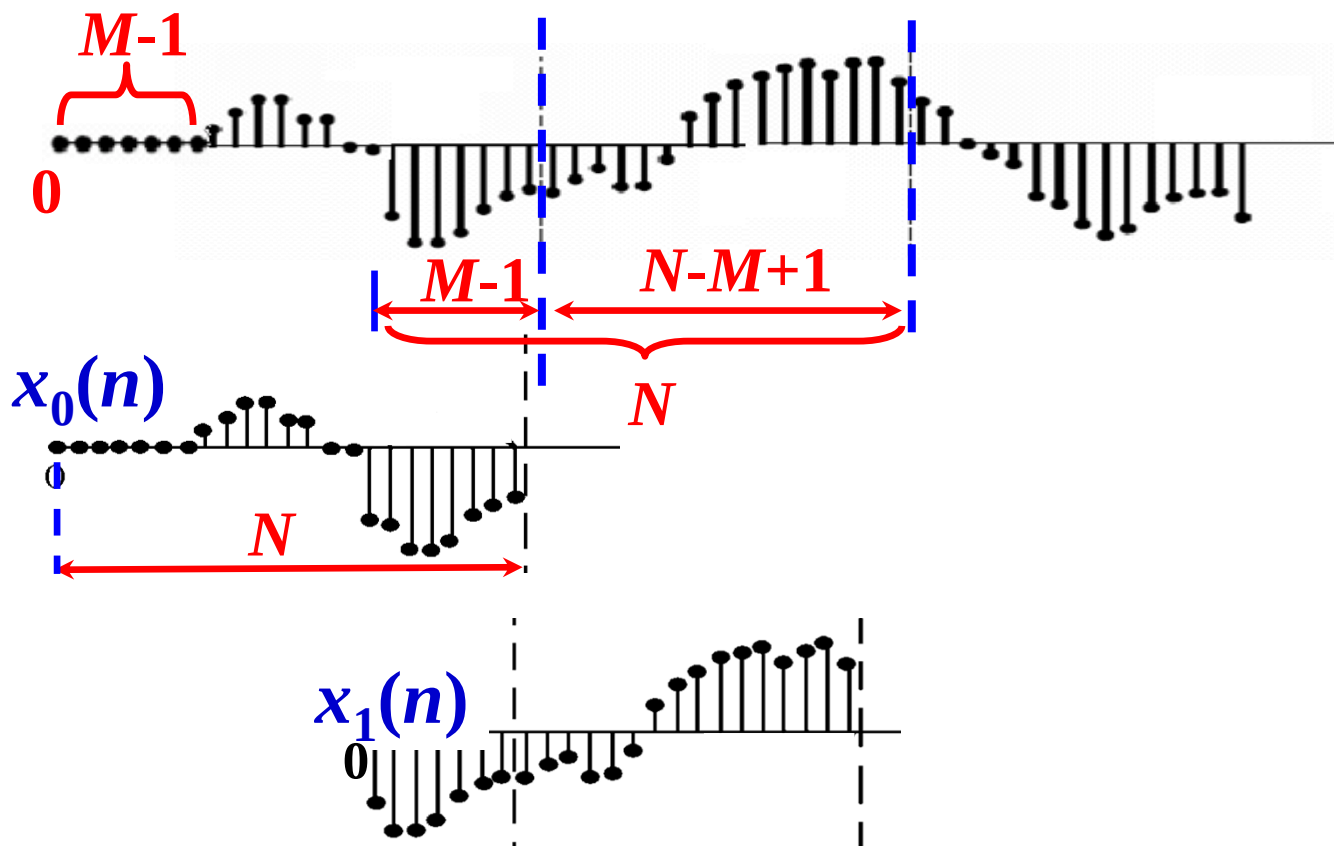
从M处开始
对应正确的
结果

➤改进措施

(2) 重叠保留法

$h(n)$ 长度为 M

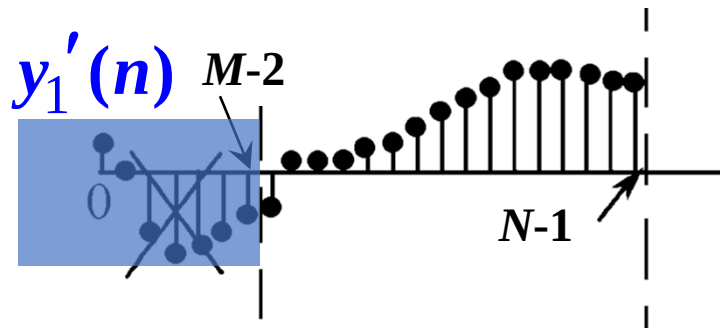
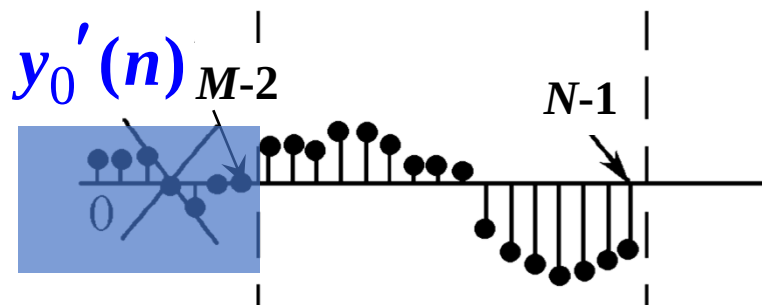
Step1: 将 $x(n)$ 分为长 N 、重叠 $M-1$ 个点的多段 $x_i(n)$



➤ 改进措施

(2) 重叠保留法

Step2: 对每一段 $x_i(n)$, 计算 $y_i'(n) = x_i(n) \circledast h(n)$



➤改进措施

(2) 重叠保留法

$h(n)$ 长度为 M

Step1: 将 $x(n)$ 分为长 N 、重叠 $M-1$ 个点的多段: $x_i(n)$

Step2: 对每一段 $x_i(n)$, 计算 $y_i'(n) = x_i(n) \circledast h(n)$

Step3: 对每一个 $y_i'(n)$, 去掉前 $M-1$ 个点, 再把其余的点衔接起来, 即得到 $y(n)$ 。

➤对比

(1) 重叠相加法

- 分段时不重叠;
- 圆周卷积不混叠;
- 各个 $y_i(n)$ 重叠相加。

(2) 重叠保留法

- 分段时有重叠;
- 圆周卷积有混叠;
- 各个 $y_i(n)$ 去掉混叠部分, 保留下来的首尾衔接。

➤改进措施

例： $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$

$h(n) = \{1, 2, -1, 1\}$ ， 用 $N=8$ 点FFT实现线性卷积，

分别用重叠相加法和重叠保留法实现。

【解答】： （1）重叠相加法

将 $x(n)$ 分为 3 段， 每段长度为 $L=5$ ：

$$x_0(n) = \{1, 2, 3, 4, 5\}_0$$

$$x_1(n) = \{6, 7, 8, 9, 10\}_5$$

$$x_2(n) = \{11, 12\}_{10}$$

➤改进措施

【解答】：（1）重叠相加法

用8点FFT实现每段 $x_i(n)$ 与 $h(n)$ 的线性卷积 $y_i(n)$ ：

$$y_0(n) = x_0(n) * h(n) = \{1, 4, 6, 9, 12, 9, -1, 5\}_0$$

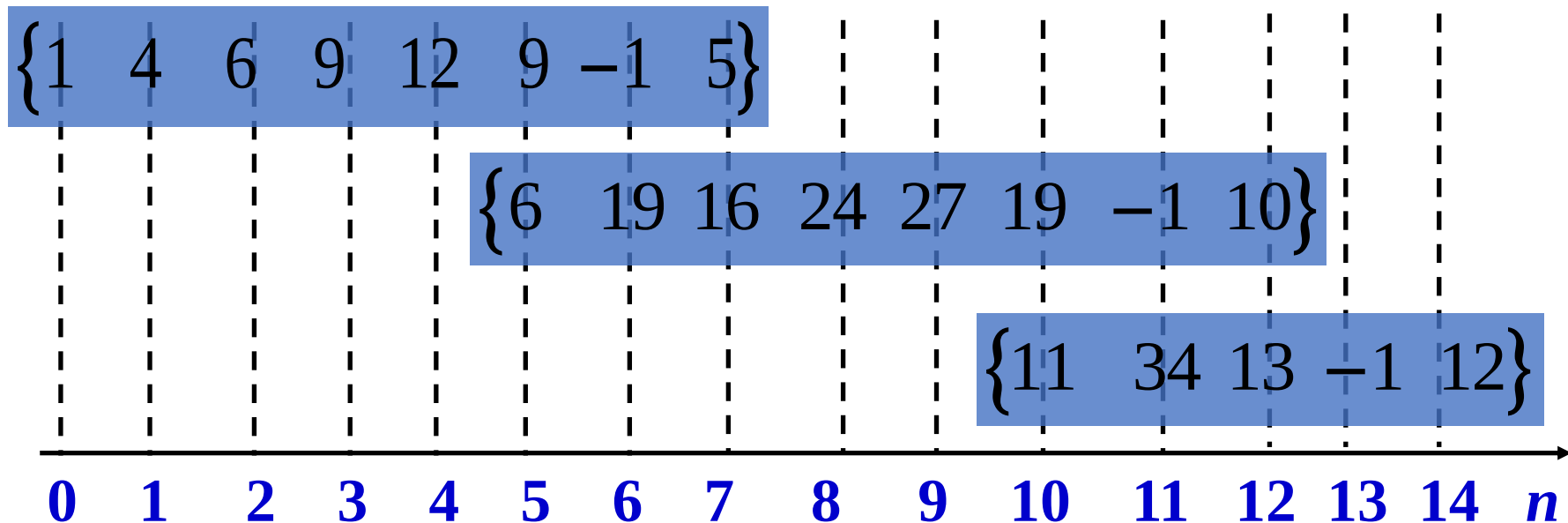
$$y_1(n) = x_1(n) * h(n) = \{6, 19, 16, 24, 27, 19, -1, 10\}_5$$

$$y_2(n) = x_2(n) * h(n) = \{11, 34, 13, -1, 12\}_{10}$$

➤改进措施

【解答】：（1）重叠相加法

将各个 $y_i(n)$ 相加：



$y(n) =$

$\{1 \ 4 \ 6 \ 9 \ 12 \ 15 \ 18 \ 21 \ 24 \ 27 \ 30 \ 33 \ 23 \ -1 \ 12\}$

➤改进措施

例： $x(n) = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$

$h(n) = \{1, 2, -1, 1\}$ ， 用 $N=8$ 点FFT实现线性卷积，

分别用重叠相加法和重叠保留法实现。

【解答】： （2）重叠保留法

将 $x(n)$ 分为 3 段，每段长度为 $L=N=8$ ，前 3 个点重叠：

$$x_0(n) = \{0, 0, 0, 1, 2, 3, 4, 5\}$$

$$x_1(n) = \{3, 4, 5, 6, 7, 8, 9, 10\}$$

$$x_2(n) = \{8, 9, 10, 11, 12\}$$

➤改进措施

【解答】：（2）重叠保留法

用 8 点FFT实现每段 $x_i(n)$ 与 $h(n)$ 的圆周卷积 $y_i(n)$ ：

$$y_0'(n) = x_0(n) \circledast h(n) = \{9, -1, 5, 1, 4, 6, 9, 12\}$$

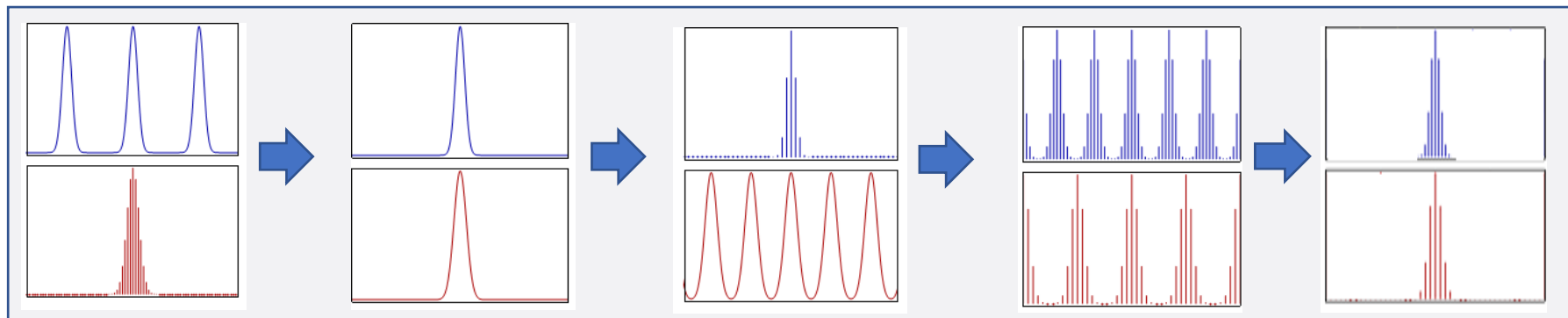
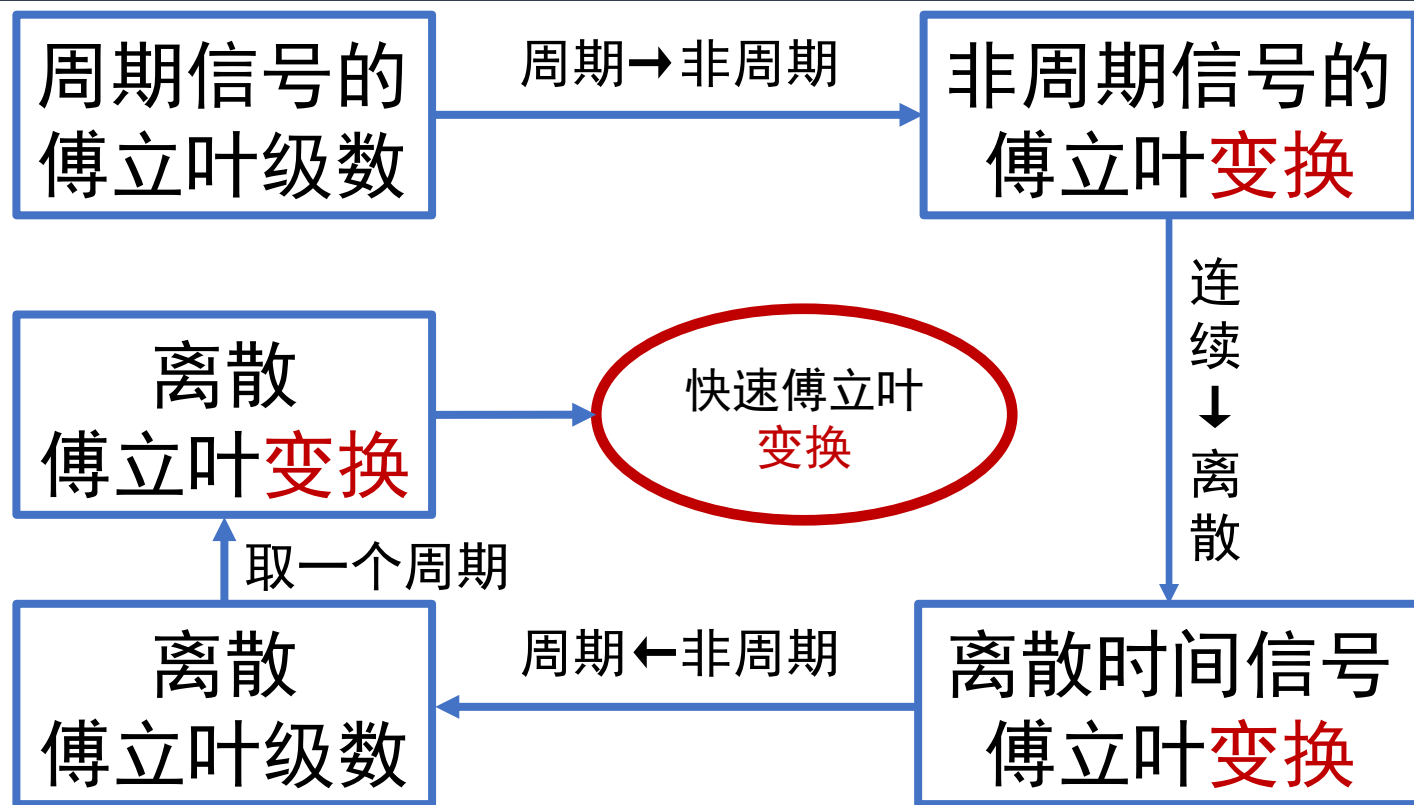
$$y_1'(n) = x_1(n) \circledast h(n) = \{22, 9, 20, 15, 18, 21, 24, 27\}$$

$$y_2'(n) = x_2(n) \circledast h(n) = \{8, 25, 20, 30, 33, 23, -1, 12\}$$

对每一个 $y_i'(n)$ ，去掉前 $M-1=3$ 个点，再把其余的点衔接起来，即得到 $y(n)$ 。

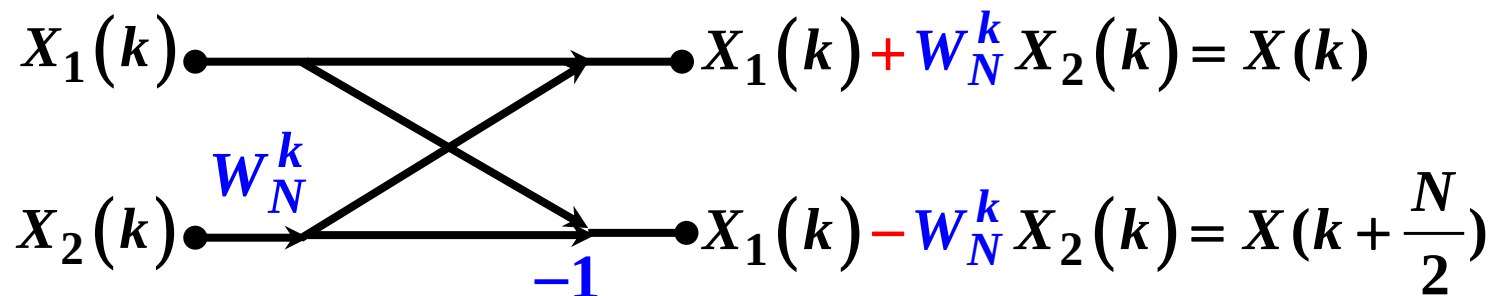
$$y(n) = \{1, 4, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 23, -1, 12\}$$

总结



$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{kn}, \quad k = 0, 1, \dots, N-1$$

$$X(k) = X_1(k) + W_N^k X_2(k)$$



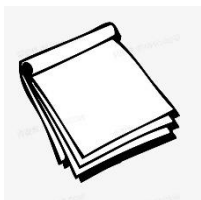
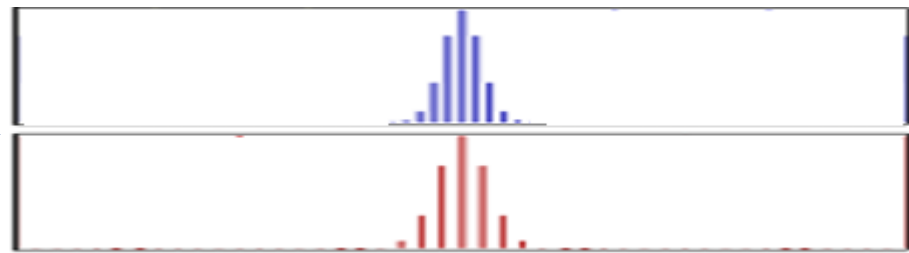
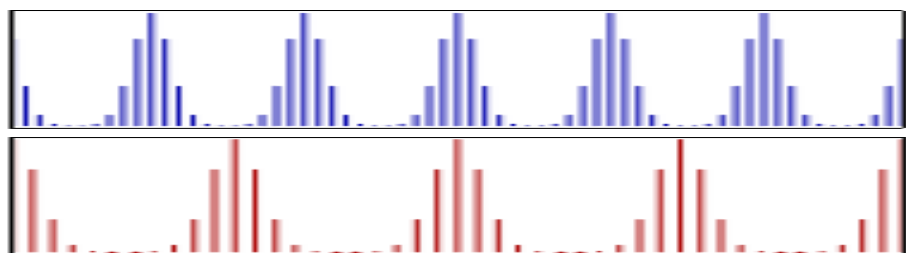
➤ DFT的优点

- 时域离散
- 频域离散
- 有限长
- 快速计算

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}, \quad k = 0, 1, \dots, N-1$$

$$X(k) = X_1(k) + W_N^k X_2(k)$$

$$N^2 \longrightarrow N \log_2 N$$



- 信号采样时，采样频率优于Nyquist频率
 - 保证可重构
- N 必须为2的整数次幂
 - 保证FFT
- 数据长度必须足够长
 - 保证频率分辨率

$$\Delta f = \frac{1}{N \cdot T_s}$$

- (1) 求 $x(n) = \cos(\frac{2\pi}{N}mn)$ 的 N 点DFT, 其中 $0 \leq n \leq N-1$, $0 < m < N$ 且 $m \in Z$ 。
- (2) 设信号 $x(t)$ 的理想抽样值序列为 $x(n)$, 数目(长度)为 L , 将这 L 个元素每 N 个一组, 其中, $N \leq L = rN + s$, $r \geq 1, s \in [0..N)$, 不足部分补零, 得到 $r+1$ 组抽样值序列分别为:

$$x_m(n) = x(mN + n), \quad n = 0, 1, \dots, N-1, \quad m = 0, 1, \dots, r$$

将上述各组序列按如下方式相加, 得到一个 N 点有限长序列

$$\tilde{x}(n) = \sum_{m=0}^r x_m(n) = \sum_{m=0}^r x(mN + n), \quad n = 0, 1, \dots, N-1$$

设 $\omega_k = k2\pi/N$, 则试证明下列等式成立:

$$\sum_{n=0}^{N-1} \tilde{x}(n)e^{-jn\omega_k} = \sum_{n=0}^{L-1} x(n)e^{-jn\omega_k}, \quad k = 0, 1, \dots, N-1$$

- (3) 设有限长序列 $x(n)$ 长度为 N , 它的 N 点DFT结果为 $X(k)$, 这里 N 是偶数。序列 $g(n)$ 是 $x(n)$ 中下标为偶数的元素组成的子序列, $h(n)$ 是 $x(n)$ 中下标为奇数的元素组成的子序列, 它们的长度是 $N/2$, 各自对应的 $N/2$ 点DFT结果分别为 $G(k)$ 和 $H(k)$ 。试根据DFT的计算公式(定义)证明:

$$X(\frac{N}{2} + k) = G(k) - W_N^k H(k), \quad k = 0, 1, \dots, N/2 - 1$$